UNIVERSIDADE PAULISTA UNIP

Projeto Integrado Multidisciplinar IV

Curso Superior de Tecnologia em

Análise e Desenvolvimento de Sistemas

LUIS FERNANDO GUSTAVO RAMOS - RA H61FHE1

RAPHAEL RODRIGO DE OLIVEIRA COSTA - RA H753DG9

JOSÉ GUILHERME FERREIRA DE SOUZA - RA F362760

FÁBIO PEREIRA BARROS - RA H75BCI9

TECHLEARN ACADEMY

Desenvolvimento e implantação de uma solução tecnológica corporativa integrada

SÃO JOSÉ DOS CAMPOS

2026

RESUMO

Este Projeto Integrado Multidisciplinar apresenta a solução tecnológica integrada da TechLearn Academy, organização fictícia de educação digital com proposta de ensino gratuito de programação. O problema estudado é a dispersão de conteúdos, matrículas e avaliações em ferramentas distintas. O objetivo é consolidar uma plataforma com aplicação web, API em C# e ASP.NET Core, banco SQL Server, desktop administrativo Windows Forms e aplicativo Android nativo em Java. A metodologia combina pesquisa bibliográfica, análise documental, inspeção dos fontes disponíveis e do pacote Android, modelagem de dados e análise dos registros de testes e da atividade de usabilidade. O desenvolvimento contempla organização, proposta de valor, viabilidade, responsabilidade social, diversidade, acessibilidade, segurança e gerenciamento ágil. A solução reúne quatro perfis, cursos por módulos, correção automática, resultados, chamados e certificados, com regras e dados centralizados na API. Os diagramas UML relacionam objetivos de uso, classes, mensagens e atividades aos fluxos implementados. O relato do Dia da UNIP registra três participantes realizando tarefas de ajuda ao mentor, avaliação e edição de perfil, com retorno favorável sobre a organização e os resultados e sugestões sobre comandos e confirmações. Esses registros constituem uma avaliação exploratória, sem medição de tempo ou validação completa de acessibilidade. As capturas reais do Android documentam acesso e painel do Aluno. Permanecem como etapas de consolidação a homologação móvel completa, a recuperação do projeto Java original e a conferência da reconstrução do desktop nativo, a avaliação com tecnologias assistivas e a validação econômica e operacional.

Palavras-chave: Educação digital. Sistemas integrados. Desenvolvimento web. Desenvolvimento mobile. Inclusão.

ABSTRACT

This Integrated Multidisciplinary Project presents the integrated technology solution for TechLearn Academy, a fictional digital education organization proposing free programming education. The problem is the distribution of content, enrollment records and assessments across separate tools. The objective is to consolidate a web application, a C# ASP.NET Core API, a SQL Server database, a Windows Forms administration client and a native Android application written in Java. The methodology combines bibliographic research, documentary analysis, inspection of available source code and of the Android package, data modeling and analysis of test and usability records. The development covers the organization, value proposition, feasibility, social responsibility, diversity, accessibility, security and agile management. The solution provides four access roles, modular courses, automatic grading, results, support tickets and certificates, with rules and data centralized in the API. UML diagrams relate user goals, classes, messages and activities to the implemented flows. Records from the Dia da UNIP event describe three participants completing mentor support, assessment and profile editing tasks. Feedback addressed the platform organization and assessment results, with suggestions about commands and confirmations. This was an exploratory evaluation without time measurements or comprehensive accessibility validation. Actual Android screenshots document the login and student dashboard. Full mobile acceptance testing, recovery of the original Java project and verification of the reconstructed desktop source, assistive technology evaluation and economic and operational validation remain consolidation activities.

Keywords: Digital education. Integrated systems. Web development. Mobile development. Inclusion.

SUMÁRIO

1 INTRODUÇÃO **5**
2 CARACTERIZAÇÃO DA ORGANIZAÇÃO **6**
2.1 Segmento, serviços e estrutura 6
2.2 Processos, problemas e oportunidades 7
3 PLANEJAMENTO DA SOLUÇÃO TECNOLÓGICA **8**
3.1 Problema, objetivos e delimitação 8
3.2 Atuação gratuita e proposta de valor 9
3.3 Viabilidade e requisitos do produto 10
4 RESPONSABILIDADE SOCIAL E DIVERSIDADE **11**
4.1 Relações étnico-raciais e afrodescendência 11
4.2 Acessibilidade, cidadania e proteção de dados 12
5 DESENVOLVIMENTO DA SOLUÇÃO WEB **13**
5.1 Arquitetura, autenticação e integração 13
5.2 APIs REST e regras de segurança 14
5.3 Estrutura administrativa e telas web 15
6 DESENVOLVIMENTO DA SOLUÇÃO MOBILE **16**
6.1 Aplicativo Android nativo em Java 16
6.2 Telas e fluxo de navegação 17
6.3 Integração e homologação do aplicativo 18
7 ARQUITETURA DE SOFTWARE E DESKTOP **19**
7.1 Camadas, modularização e orientação a objetos 19
7.2 Aplicação desktop administrativa 20

SUMÁRIO

7.3 Correspondência entre código e comportamento 21

8 PROJETO DO BANCO DE DADOS 22

8.1 Modelo entidade-relacionamento acadêmico 22

8.2 Modelo de autenticação e atendimento 23

8.3 Modelo de certificados e mentoria 24

8.4 Modelo lógico e dicionário do núcleo acadêmico 25

8.5 Modelo lógico de resultados e suporte 26

8.6 Procedures, triggers e transações 27

8.7 Modelo físico, otimização e script completo 28

9 INFRAESTRUTURA EM NUVEM E DEVOPS 29

9.1 Arquitetura, serviços e segurança operacional 29

9.2 Containers, pipeline, monitoramento e recuperação 30

10 GERENCIAMENTO ÁGIL E VALIDAÇÃO 31

10.1 Product Backlog e priorização 31

10.2 Sprint Backlog de consolidação 32

10.3 Cronograma, Kanban e entregas 33

10.4 Evidências, testes e rastreabilidade 34

10.5 Testes de usabilidade no Dia da UNIP 36

11 CONCLUSÃO 37

REFERÊNCIAS 38

1 INTRODUÇÃO

A transformação digital de uma organização educacional exige que conteúdo, acesso e acompanhamento do aluno funcionem como um processo integrado. Na TechLearn Academy, empresa fictícia adotada no projeto, a distribuição de arquivos e o controle manual de matrículas e notas representam o cenário inicial de análise. O PIM IV reorganiza o planejamento preliminar do grupo e reúne os componentes tecnológicos necessários à administração e à aprendizagem em um ambiente compartilhado.

O objetivo geral é desenvolver e documentar uma solução educacional de acesso gratuito que integre aplicação web, API em .NET, banco SQL Server, desktop administrativo e aplicativo Android nativo em Java. Os objetivos específicos são centralizar registros, controlar o acesso, automatizar avaliações, preservar o histórico e planejar implantação, inclusão e responsabilidade social.

A pesquisa é aplicada, descritiva e documental, com apoio bibliográfico (GERHARDT; SILVEIRA, 2009). Foram examinados planejamento, fontes disponíveis, scripts SQL, pacotes distribuídos e registros de teste. A análise também incorpora o relato de observação de três participantes no Dia da UNIP. Não houve pesquisa de mercado; o custeio proposto e as semanas de planejamento são hipóteses acadêmicas. As conclusões distinguem inspeção documental, observação de uso e testes ainda pendentes.

O trabalho articula Gerenciamento de Projetos Ágil, Desenvolvimento Web com .NET, Programação de Banco de Dados, Desenvolvimento Mobile, Desenvolvimento Mobile em JavaScript, Cloud Computing and DevOps, Empreendedorismo em TI e Relações Étnico-Raciais e Afrodescendência. Orientação a objetos e organização em camadas são tratadas como fundamentos transversais da solução.

Os capítulos seguintes relacionam negócio, arquitetura e validação. A entrega móvel examinada é um APK Android em Java, acompanhado de capturas de acesso e painel; o aceite dos demais fluxos exige registros específicos. O Apêndice A contém o SQL integral; o Apêndice B detalha a UML; o Apêndice C reúne o roteiro e as evidências de usabilidade; o Apêndice D registra rastreabilidade e pendências de aceite.

2 CARACTERIZAÇÃO DA ORGANIZAÇÃO

2.1 Segmento, serviços e estrutura

A TechLearn Academy é uma organização fictícia do segmento de tecnologia educacional dedicada à formação em programação e desenvolvimento de sistemas. A oferta é predominantemente assíncrona: o estudante acessa materiais e avaliações em horários compatíveis com sua rotina. Sua identidade reúne tecnologia, aprendizagem e organização acadêmica, sem pressupor a existência de uma empresa formalmente constituída ou de clientes reais.

Os serviços centrais são cursos organizados em módulos, materiais em PDF e links para videoaulas, matrícula, avaliações objetivas e consulta de resultados. O código atual também contempla mentoria por chamados, ouvidoria, relatórios e solicitação de certificados. Essas extensões ampliam a jornada inicialmente descrita nos documentos de caracterização e planejamento.

Quadro 1 - Estrutura organizacional funcional

Área	Responsabilidade na organização
Administração	Define prioridades, acompanha custos e estabelece regras de operação e atendimento.
Produção de conteúdo	Elabora cursos, revisa materiais e gabaritos e verifica linguagem e representação dos exemplos.
Tecnologia	Mantém aplicações, API, banco de dados, segurança, implantação e suporte técnico.

Fonte: elaboração dos autores (2026).

A estrutura do negócio se distingue dos perfis do sistema. Na aplicação, Aluno estuda; Mentor acompanha chamados de aprendizagem; Gestor mantém conteúdos, relatórios e certificados; Administrador gerencia contas e atende demandas escaladas. Uma mesma área organizacional pode reunir pessoas com responsabilidades diferentes, mas cada conta mantém o perfil definido no cadastro.

A operação depende de conteúdo tecnicamente correto e de registros confiáveis. A administração coordena a oferta, a equipe de conteúdo verifica a qualidade pedagógica e a tecnologia assegura que as permissões sejam aplicadas no servidor. Essa separação reduz conflitos entre a organização das aulas e a manutenção da infraestrutura.

2.2 Processos, problemas e oportunidades

No cenário inicial, a produção de conteúdo organiza arquivos e links; a administração registra o aluno e o curso em planilhas; as respostas são conferidas manualmente e os resultados são comunicados por mensagens. Trata-se do cenário de negócio construído para o estudo, e não de observação empírica de uma instituição em funcionamento.

Quadro 2 - Diagnóstico dos processos

Processo	Problema inicial	Resposta tecnológica
Oferta de cursos	Arquivos dispersos e versões divergentes.	Curso, módulo e material relacionados no banco.
Matrícula	Duplicidade e conferência manual.	Vínculo autenticado e unicidade de matrícula ativa.
Avaliação	Correção lenta e risco de erro.	Gabarito no servidor e resultado transacional.
Acompanhamento	Notas separadas do conteúdo.	Histórico de tentativas e progresso por módulo.
Atendimento	Dúvidas perdidas em mensagens.	Fila de chamados, responsável e conversa preservada.

Fonte: elaboração dos autores (2026).

O processo integrado começa quando o Gestor cadastra o curso, organiza até quatro módulos, associa materiais e prepara avaliações. O Aluno consulta o catálogo, confirma a matrícula, estuda e envia suas respostas. A API verifica a matrícula, corrige a avaliação, registra a tentativa e conclui o módulo quando todas as suas avaliações ativas estão aprovadas. O resultado alimenta a consulta do próprio aluno e os relatórios autorizados.

A oportunidade de melhoria é centralizar a informação e tornar cada transição verificável. Um material pertence a um módulo; uma matrícula relaciona aluno e curso; uma resposta pertence a uma questão da avaliação enviada. As chaves e regras de negócio evitam que dados aparentemente válidos sejam associados ao contexto errado.

O ganho esperado é reduzir retrabalho e facilitar o acompanhamento. Para medir esse ganho em um piloto, o projeto propõe comparar tempo de matrícula, tempo até o resultado e quantidade de registros corrigidos. Sem uma linha de base observada, o trabalho não atribui percentuais de redução ou aumento de aprendizagem à plataforma.

3 PLANEJAMENTO DA SOLUÇÃO TECNOLÓGICA

3.1 Problema, objetivos e delimitação

A solução trata a ausência de um ambiente único para administrar conteúdos e acompanhar o aluno. O público inicial é composto por estudantes, iniciantes em tecnologia, autodidatas e pessoas em transição profissional que precisam conciliar estudo e outras atividades. O acesso assíncrono, os materiais organizados e a consulta de progresso compõem a proposta de valor.

O objetivo do produto é permitir que uma pessoa se cadastre, encontre um curso, realize sua matrícula, acesse conteúdos e consulte um resultado coerente com as respostas enviadas. Para a organização, o mesmo fluxo deve oferecer manutenção dos conteúdos, gestão de contas e acompanhamento operacional sem alterações manuais de dados por usuários finais.

Quadro 3 - Evolução do escopo

Aspecto	Escopo inicial	Versão atual examinada
Perfis	Administrador e Aluno.	Aluno, Mentor, Gestor e Administrador.
Conteúdo	Cursos, módulos, PDF e links.	Gestão por Gestor e envio de PDF de até 10 MB.
Acompanhamento	Resultados e progresso.	Chamados, relatórios e orientações.
Conclusão	Sem certificados.	Solicitação e aprovação de certificado.
Canais	Web e mobile planejado.	Web, desktop nativo e aplicativo Android em Java.

Fonte: elaboração dos autores (2026).

O Product Backlog elaborado durante a Sprint 2 delimita o MVP inicial e exclui chat, mentoria e certificados. O código posterior incorpora essas funções. O relatório trata essa diferença como evolução de escopo, mantendo a versão inicial como referência para avaliar as alterações do produto.

Pagamentos integrados, streaming próprio, aulas ao vivo e inteligência artificial permanecem fora da solução descrita. O APK Android nativo utiliza a mesma API e as mesmas regras do servidor usadas pelo cliente web. Seu pacote distribuído foi inspecionado; o capítulo 6 distingue os recursos encontrados no APK dos testes que ainda dependem de dispositivo.

3.2 Atuação gratuita e proposta de valor

Laverty e Littel (2020) relacionam o modelo de negócio ao público, à proposta de valor, à operação e às receitas. A TechLearn Academy propõe acesso gratuito a cursos, materiais, avaliações e certificados, para reduzir barreiras financeiras à formação em tecnologia e dar continuidade ao projeto acadêmico como iniciativa educacional voltada à comunidade.

Quadro 4 - Organização da oferta educacional gratuita

Componente da oferta	Definição para a TechLearn Academy
Público beneficiário	Iniciantes, estudantes e pessoas em aperfeiçoamento ou transição para tecnologia.
Proposta de valor	Cursos gratuitos por módulos, estudo assíncrono, correção automática e acompanhamento centralizado.
Canais	Site responsivo; aplicativo móvel; divulgação institucional e comunidades de estudo.
Relacionamento	Autoatendimento, ajuda com mentor e ouvidoria encaminhada ao Gestor.
Acesso e custeio	Sem cobrança ao aluno; proposta de custeio inicial pela equipe, com orçamento a definir.
Recursos principais	Conteúdo, equipe, código, banco, infraestrutura e documentação.
Atividades principais	Produzir conteúdo, manter a solução, atender chamados e acompanhar indicadores.
Parcerias possíveis	Autores e instituições; apoio futuro depende de negociação, sem compromisso confirmado.
Custos principais	Produção de conteúdo, manutenção, hospedagem, banco, comunicação e suporte.

Fonte: elaboração dos autores (2026).

A gratuidade é uma escolha de atuação: o aluno não pagará matrícula, mensalidade ou acesso aos recursos educacionais. O valor oferecido combina conteúdo progressivo, retorno imediato da avaliação e suporte por curso. A observação exploratória com três participantes no Dia da UNIP oferece indícios de compreensão desses recursos, sem comprovar aprendizagem duradoura ou capacidade de atendimento em escala.

A disponibilização pública será gradual e condicionada à capacidade da equipe. O piloto acompanhará conclusão de módulos, retorno ao estudo, chamados e custos. A expansão dependerá desses indicadores e dos recursos disponíveis; a proposta de gratuidade não elimina o planejamento da operação.

3.3 Viabilidade e requisitos do produto

A API e o banco compartilhados favorecem a viabilidade técnica; conteúdo, atendimento e administração exigem trabalho contínuo. Para examinar a gratuidade, simula-se um piloto pequeno com teto mensal de R$ 350,00. Os valores do Quadro 5 são premissas didáticas, sem cotação de serviços, compromisso de aporte ou comprovação de capacidade. O Product Owner organiza o levantamento; o grupo deve aprovar o orçamento antes da abertura pública.

Quadro 5 - Simulação mensal de custeio do piloto gratuito

Item	Hipótese mensal	Premissa do cenário
Hospedagem web e API	R$ 120,00	Uma instância para o piloto; preço a cotar.
Banco de dados	R$ 120,00	Armazenamento e uso inicial; preço a cotar.
Domínio e e-mail	R$ 20,00	Estimativa mensalizada; preço a cotar.
Backup e monitoramento	R$ 40,00	Rotina de proteção e alertas; preço a cotar.
Reserva para variação	R$ 50,00	Margem hipotética para custos não previstos.
Total mensal simulado	R$ 350,00	Limite de referência; financiamento não confirmado.

Fonte: elaboração dos autores (2026).

A soma simulada é de R$ 350,00 por mês, incluindo R$ 50,00 de reserva. Essa quantia não contempla remuneração da equipe, produção inicial de conteúdo, tributos ou crescimento da demanda. O custeio pela equipe ainda depende de decisão coletiva; sem recursos confirmados, a abertura pública deve permanecer limitada ao piloto. Parcerias ou doações são alternativas futuras, sem acordo registrado.

Quadro 6 - Requisitos centrais e critérios

Requisito	Critério verificável
RF01 Acesso	Cadastro público cria Aluno e login respeita o perfil.
RF02 Conteúdo	Gestor mantém curso, módulo, material e avaliação.
RF03 Aprendizagem	Aluno matriculado responde e consulta o próprio resultado.
RF04 Atendimento	Chamado tem solicitante, responsável e histórico.
RNF01 Segurança	API verifica sessão, perfil e contexto de cada recurso.
RNF02 Usabilidade	Rótulos, foco e mensagens são compreensíveis em tela pequena.

Fonte: elaboração dos autores (2026).

4 RESPONSABILIDADE SOCIAL E DIVERSIDADE

4.1 Relações étnico-raciais e afrodescendência

A inclusão digital envolve condições de acesso, permanência e reconhecimento. Em uma plataforma de formação tecnológica, a ausência de barreiras explícitas não basta para assegurar participação: a linguagem, os exemplos e as formas de atendimento também podem reproduzir discriminação. O Estatuto da Igualdade Racial constitui referência para a discussão de oportunidades educacionais e valorização da população negra (BRASIL, 2010).

A política proposta para a TechLearn Academy proíbe discriminação racial nos conteúdos e nas interações. A produção de aulas deve revisar expressões, exemplos e representações, contextualizar a participação de profissionais negros em tecnologia e evitar estereótipos sobre aptidão ou desempenho. Essas ações são diretrizes editoriais; sua existência no relatório não significa que todo o catálogo tenha recebido auditoria especializada.

Quadro 7 - Ações de diversidade e responsabilidade social

Ação proposta	Responsável funcional	Forma de acompanhamento
Revisar linguagem e exemplos	Produção de conteúdo	Lista editorial antes da liberação de cada curso.
Dar visibilidade a trajetórias negras	Produção de conteúdo	Referências identificadas e contextualizadas nas aulas.
Acolher denúncias de discriminação	Gestor	Registro em ouvidoria e tratamento reservado.
Preparar atendimento respeitoso	Administração	Orientações e revisão de casos sem expor alunos.
Avaliar barreiras de participação	Administração e tecnologia	Feedback voluntário e análise de dificuldades de uso.

Fonte: elaboração dos autores (2026).

A ouvidoria já existente oferece um canal tecnológico que pode apoiar o acolhimento. O procedimento de tratamento proposto compreende receber o relato, restringir o acesso aos responsáveis, preservar evidências, ouvir as partes e registrar a decisão. Prazos de apuração e sanções precisam ser definidos pela organização antes de uma operação real.

Não se deve inferir raça por nome, fotografia ou localização. Indicadores de diversidade exigem finalidade clara e proteção dos dados. O sistema não contém pesquisa racial; a avaliação social deve respeitar esse limite.

4.2 Acessibilidade, cidadania e proteção de dados

O acesso inclusivo requer que informações sejam perceptíveis, controles possam ser operados e mensagens sejam compreensíveis. As WCAG 2.2 orientam a verificação por critérios testáveis (W3C, 2024). A interface responsiva, os rótulos e o foco visível encontrados no projeto contribuem para esse objetivo, mas a conformidade completa depende de avaliação com tecnologias assistivas.

Quadro 8 - Plano de acessibilidade

Dimensão	Aplicação no projeto	Validação necessária
Percepção	Textos legíveis, contraste e instruções textuais.	Medir contraste e conferir alternativas para conteúdo visual.
Operação	Foco e sequência de navegação por teclado.	Percorrer todos os fluxos sem mouse.
Compreensão	Rótulos, mensagens de erro e confirmação.	Testar formulários com pessoas do público inicial.
Conteúdo inclusivo	PDF legível, vídeo com alternativas acessíveis.	Conferir estrutura de PDF, legendas e transcrições.
Mobile	Reorganização do conteúdo em telas pequenas.	Leitor de tela, ampliação e orientação do dispositivo.

Fonte: elaboração dos autores (2026).

O site carrega o widget VLibras para tradução automática do texto das páginas, dependente de conexão com o serviço externo. Isso não fornece legendas aos vídeos externos nem traduz automaticamente os PDFs. A curadoria deverá verificar a acessibilidade de cada material e oferecer alternativas equivalentes; ainda falta validação com usuários de tecnologias assistivas.

A cidadania digital é apoiada pelo acesso à formação, pela transparência dos resultados e por um canal de contestação. A plataforma deve permitir que o aluno entenda como a nota foi calculada e obtenha orientação sobre dificuldades, sem exposição pública de seu desempenho.

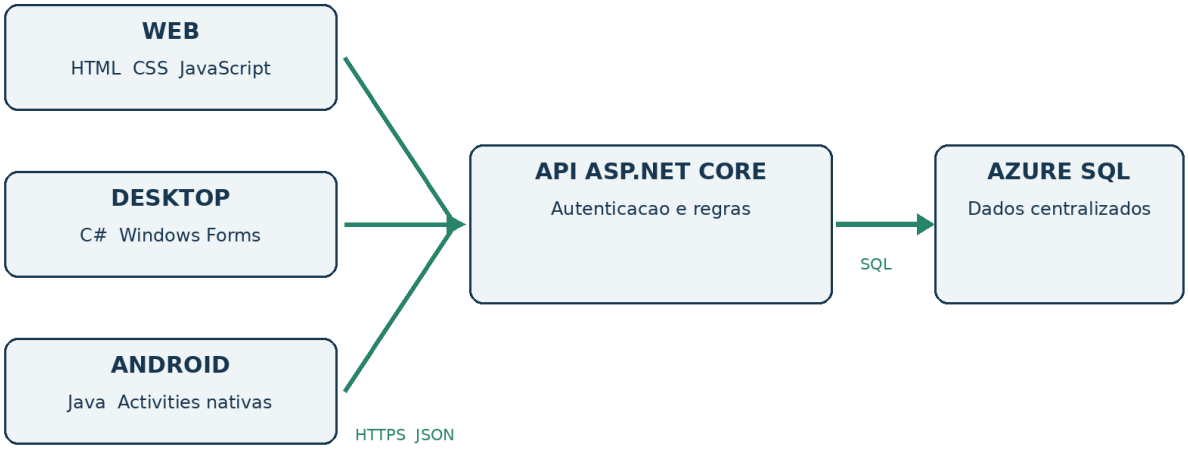
A coleta observada inclui nome, e-mail, telefone com DDD e gênero. Telefone e gênero são obrigatórios no cadastro web e validados pela API; o SQL permite nulos para preservar contas antigas. O código usa e-mail no login e na recuperação, mas não demonstra uso de telefone ou gênero em atendimento, avaliação ou relatórios. Contato alternativo e análise voluntária de inclusão são finalidades possíveis, ainda não aprovadas nem implementadas. Antes da abertura pública, o grupo deve justificar cada campo ou tornar sua coleta opcional, além de definir acesso, retenção e atendimento aos titulares (BRASIL, 2018).

5 DESENVOLVIMENTO DA SOLUÇÃO WEB

5.1 Arquitetura, autenticação e integração

A stack entregue usa HTML, CSS e JavaScript na web; C# com ASP.NET Core e .NET 10 na API; SQL Server no banco; C# com Windows Forms no desktop distribuído; e Java com Activities no APK Android. Web e API estão no Azure App Service e usam Azure SQL. Cada interface acessa os mesmos dados pela API.

Figura 1 - Componentes e comunicação da solução



Fonte: elaboração dos autores (2026).

Program.cs registra os serviços e configura autenticação, autorização, tratamento de erros e arquivos estáticos. Controllers recebem requisições; serviços coordenam regras; ConexaoBanco executa consultas com parâmetros por Microsoft.Data.SqlClient. A separação se apoia nos mecanismos de injeção de dependências e processamento HTTP do ASP.NET Core (MICROSOFT, s.d.a).

O login protege a senha com PBKDF2-HMAC-SHA256, sal aleatório e 600 mil iterações. A API gera um token de sessão e guarda somente seu hash. Na web, envia cookie HttpOnly, SameSite Strict e Secure em HTTPS; o Android apresenta a credencial em Authorization: Bearer. SessaoHandler consulta validade, revogação e conta ativa e produz a identidade usada na autorização. Não se usa JWT. O token também é retornado no JSON do login; o JavaScript examinado não o persiste.

Os clientes trocam HTTPS e JSON: a web usa fetch; o Android, HttpURLConnection; o desktop nativo, um cliente HTTP. O pipeline autentica e autoriza; controller, serviço e Microsoft.Data.SqlClient processam SQL parametrizado. A API devolve o resultado, sem fornecer credenciais SQL aos clientes. O cadastro também valida telefone e gênero no servidor.

5.2 APIs REST e regras de segurança

Quadro 9 - Contratos principais da API atual

Método e rota principal	Uso e autorização
POST /api/usuarios/cadastro	Cadastro público; a conta criada é Aluno.
POST /api/usuarios/login	Valida credenciais e cria sessão real.
GET /api/cursos	Consulta por Aluno ou Gestor autenticado.
POST /api/cursos	Criação exclusiva de Gestor.
POST /api/matriculas	Matrícula do próprio Aluno.
GET /api/modulos/{id}/materiais	Gestor ou Aluno com matrícula válida.
POST /api/avaliacoes/{id}/respostas	Aluno; corrige e grava tentativa no servidor.
GET /api/resultados/{id}	Dono, Mentor autorizado ou Gestor, conforme regra.
GET /api/administracao/usuarios	Gestor ou Administrador, com filtro por contexto.
POST /api/chamados/{id}/assumir	Perfil de atendimento e chamado elegível.
POST /api/certificados/{id}/decisao	Gestor; confere requisitos antes de emitir.

Fonte: elaboração dos autores (2026).

As respostas usam códigos HTTP para expressar o resultado: 201 para criação, 400 para dados inválidos, 401 para ausência ou invalidade de sessão, 403 para permissão insuficiente, 404 para recurso não encontrado e 409 para conflito. A mensagem de erro é devolvida em JSON no campo mensagem. O inventário eletrônico do repositório reúne as rotas e os atributos de autorização extraídos dos controllers.

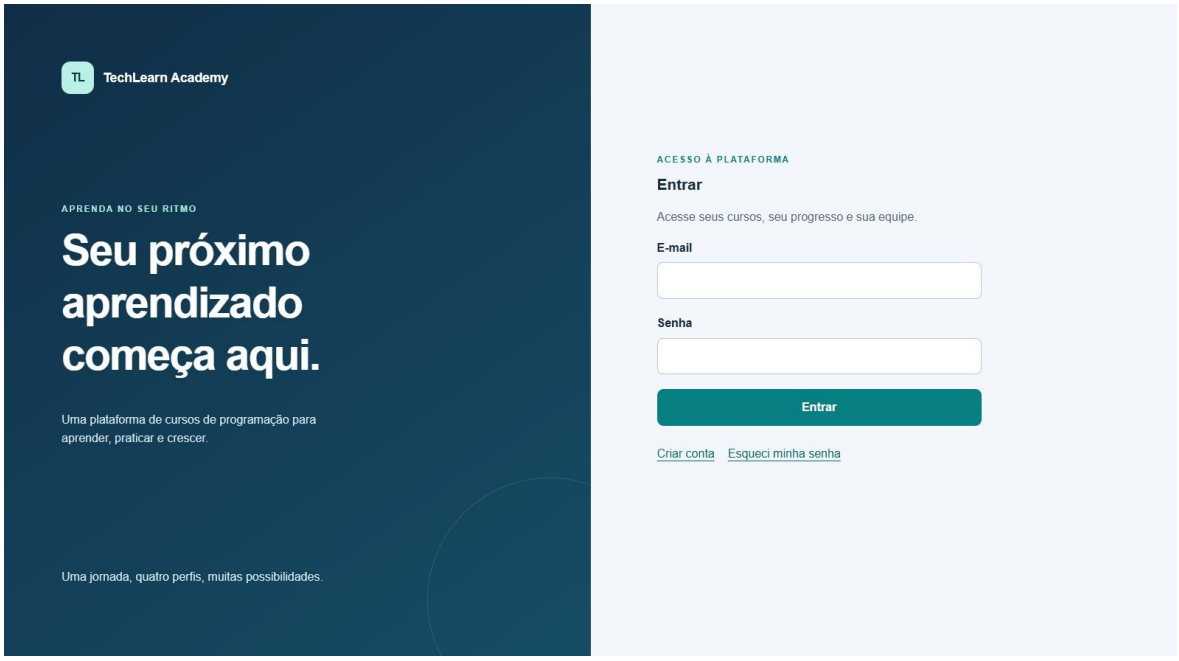
As rotas de usuários cuidam de login, logout, recuperação e redefinição de senha. ASP.NET Core usa SessaoHandler e autorização por perfil; System.Security.Cryptography fornece hashes e aleatoriedade. SQL parametrizado, verificação de origem em gravações, CORS configurável, no-store, nosniff e HTTPS complementam a proteção. A revisão de produção ainda requer testes específicos contra requisições forjadas e tentativas repetidas de login.

A senha exige pelo menos oito caracteres, maiúscula, número e símbolo, sem espaços. A recuperação usa token de uso único válido por 30 minutos. Na publicação, o envio por Azure Communication Services Email e a redefinição foram confirmados em 18 de outubro de 2026; SMTP é alternativa local. O Administrador não dispõe de recuperação pelo site.

5.3 Estrutura administrativa e telas web

A estrutura administrativa possui responsabilidades separadas. O Gestor mantém cursos, módulos, materiais, avaliações, relatórios e certificados. O Administrador gerencia contas e a situação de acesso. Mentor recebe demandas de aprendizagem; Aluno utiliza os cursos e resultados. A apresentação de menus acompanha o perfil, mas a proteção efetiva permanece nos endpoints.

Figura 2 - Tela web de autenticação



Fonte: acervo de testes do projeto (2026).

O login oferece acesso, cadastro público e recuperação. Após autenticação, o usuário é encaminhado ao painel correspondente. A navegação do aluno reúne catálogo, cursos e resultados; o Gestor encontra funções de conteúdo e acompanhamento; o Administrador encontra usuários e chamados. Listas vazias recebem mensagens explícitas para distinguir ausência de dados de falha de carregamento.

O cadastro de PDF utiliza envio multipart e limite de 10 MB para o arquivo. A recuperação do material é feita por endpoint autorizado. Links de videoaulas continuam externos; o sistema não hospeda streaming próprio. Uma exclusão de conteúdo remove sua disponibilidade nas telas e preserva registros históricos por marcação lógica, respeitando as regras da aplicação.

6 DESENVOLVIMENTO DA SOLUÇÃO MOBILE

6.1 Aplicativo Android nativo em Java

A entrega móvel é TechLearnAcademy-Android.apk, um aplicativo Android nativo em Java. A inspeção registrada identificou AndroidManifest.xml, LoginActivity, PainelActivity, CursosActivity, AvaliacaoActivity e PdfActivity, além de TextView, EditText, Button e ScrollView. Activities organizam a interação com o usuário no Android (GOOGLE, s.d.a). O APK distribuído não usa a WebView para desenhar suas telas.

O pacote identifica Api, Sessao, Validacao e TelaBase para comunicação, sessão e controles comuns. O LEIA-ME descreve seus arquivos Java, que não acompanham a entrega. JavaScript permanece na aplicação web, com eventos, validações e fetch; não é apresentado como linguagem da interface Android atual. O projeto MAUI pertence a uma versão anterior e não caracteriza este APK.

Quadro 10 - Organização do componente mobile

Componente	Responsabilidade
Activities Java	Telas nativas por perfil e navegação.
Api	Solicitações HTTPS, JSON, arquivos e token de sessão.
Sessao e Validacao	Estado de sessão e validação de campos, conforme o pacote.
TelaBase e manifesto	Controles comuns, registro de telas e permissão de internet.
API e Azure SQL	Autenticação, autorização, regras e persistência.

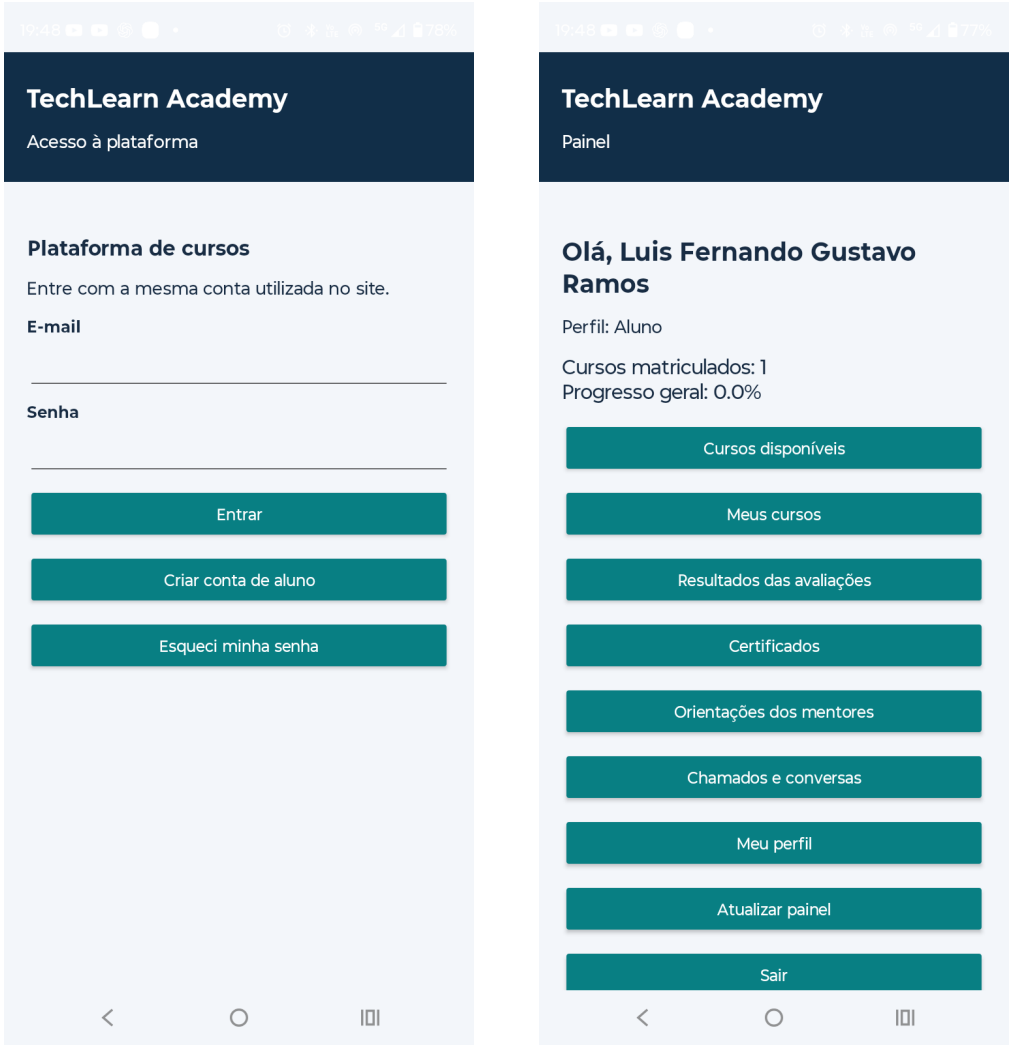
Fonte: elaboração dos autores (2026).

O manifesto inclui android.permission.INTERNET e o binário referencia a URL HTTPS pública e rotas da plataforma. O material da entrega descreve sessão em memória, PDFs com PdfRenderer e PdfDocument e links externos. A presença dessas referências não comprova todos os caminhos de execução ou ausência de armazenamento local de credenciais.

A análise anterior registrou APK de 100.692 bytes, integridade ZIP e bloco de assinatura v2. Essas verificações não substituem a validação criptográfica completa, a execução dos fluxos no aparelho ou a revisão dos fontes Java correspondentes.

6.2 Telas e fluxo de navegação

Figura 3 - Telas reais de acesso e painel do Aluno no Android



Fonte: capturas do aplicativo Android instalado, fornecidas pelo grupo (2026).

O Aluno percorre login, painel, catálogo, detalhes, matrícula, módulos, materiais, avaliação e resultado. As capturas mostram acesso e painel; não comprovam a execução completa dessa jornada. O pacote também identifica áreas de chamados, mentoria, relatórios, administração e certificados, condicionadas ao perfil e às permissões da API.

Os contratos são compartilhados com a web e o desktop. A API confere sessão, matrícula e regras acadêmicas, inclusive a aprovação mínima de 60%. A tela organiza a interação; a autorização e o cálculo válido permanecem no servidor.

6.3 Integração e homologação do aplicativo

A comunicação usa HTTPS e JSON, com referências a HttpURLConnection, Authorization e Bearer no pacote. A sessão é conferida pela API. Operações de rede devem evitar bloqueio da interface Android (GOOGLE, s.d.b); o uso descrito de Thread e runOnUiThread depende de revisão dos fontes Java para conferência completa.

A sincronização é online: cada alteração exige confirmação do servidor. Não foi documentada fila offline. Em falha de rede, o cliente deve informar o erro e consultar o estado antes de repetir um envio incerto. O logout deve revogar a sessão e limpar seu estado local; esses comportamentos exigem evidência de execução.

Quadro 11 - Roteiro de homologação do aplicativo

ID	Verificação de aceite	Evidência necessária
M01	Instalar e abrir; conferir perfil.	Capturas de acesso e painel disponíveis; registrar aparelho e Android.
M02	Entrar, sair e verificar recusa.	Sessão revogada e acesso por perfil; pendente de relatório.
M03	Catálogo, matrícula e módulos.	Estado compartilhado com a API; pendente de relatório.
M04	PDF protegido e retorno.	Leitura autorizada e navegação; pendente de relatório.
M05	Avaliação e resultado.	Nota, acertos e progresso coerentes; pendente de relatório.
M06	Perda de rede e sessão vencida.	Erro claro e ausência de sucesso fictício; pendente de relatório.

Fonte: elaboração dos autores (2026).

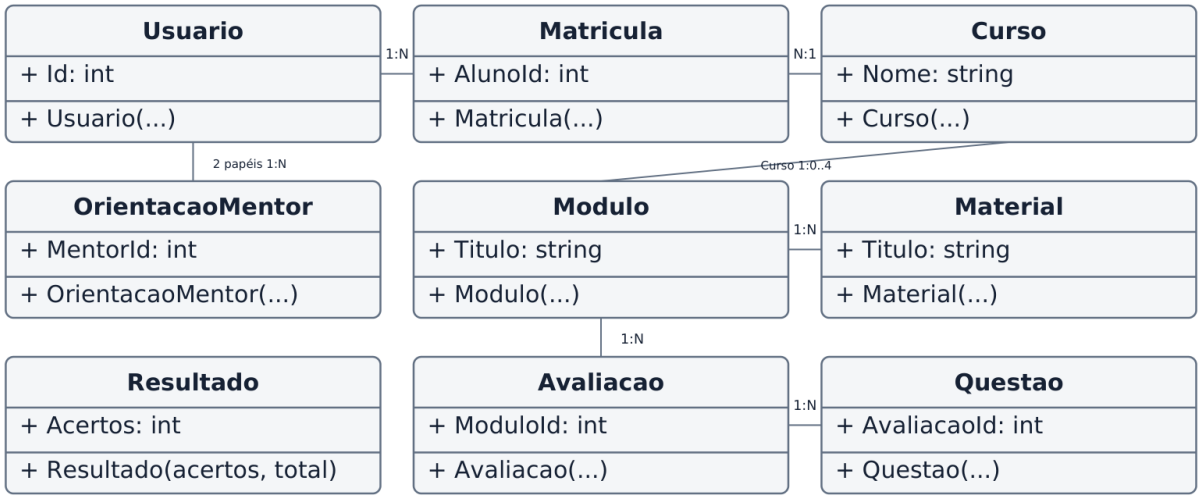
O LEIA-ME relata integração, cinco verificações Robolectric e Android Lint sem erros, mas seus relatórios não acompanham o pacote. As capturas fornecidas documentam duas telas reais. O aceite completo depende dos casos M01 a M06 com data, dispositivo, versão, resultado e evidência; esses ensaios não foram reexecutados nesta consolidação.

7 ARQUITETURA DE SOFTWARE E DESKTOP

7.1 Camadas, modularização e orientação a objetos

Valente (2020) apresenta a arquitetura como organização de componentes e dependências relevantes. Na TechLearn Academy, apresentação, serviços e persistência compõem um monólito modular: uma API reúne usuários, cursos, avaliações e gestão. Essa separação concentra as regras e permite compartilhar os mesmos serviços entre web, mobile e desktop.

Figura 4 - Nove classes centrais com membros selecionados



Fonte: elaboração dos autores (2026).

As nove classes são Usuario, Curso, Modulo, Material, Matricula, Avaliacao, Questao, Resultado e OrientacaoMentor. A figura apresenta um recorte; o Apêndice B detalha propriedades, operações e vínculos. As estruturas Dados de Pedidos.cs recebem requisições HTTP e não são subclasses dessas classes nem reproduções completas das tabelas.

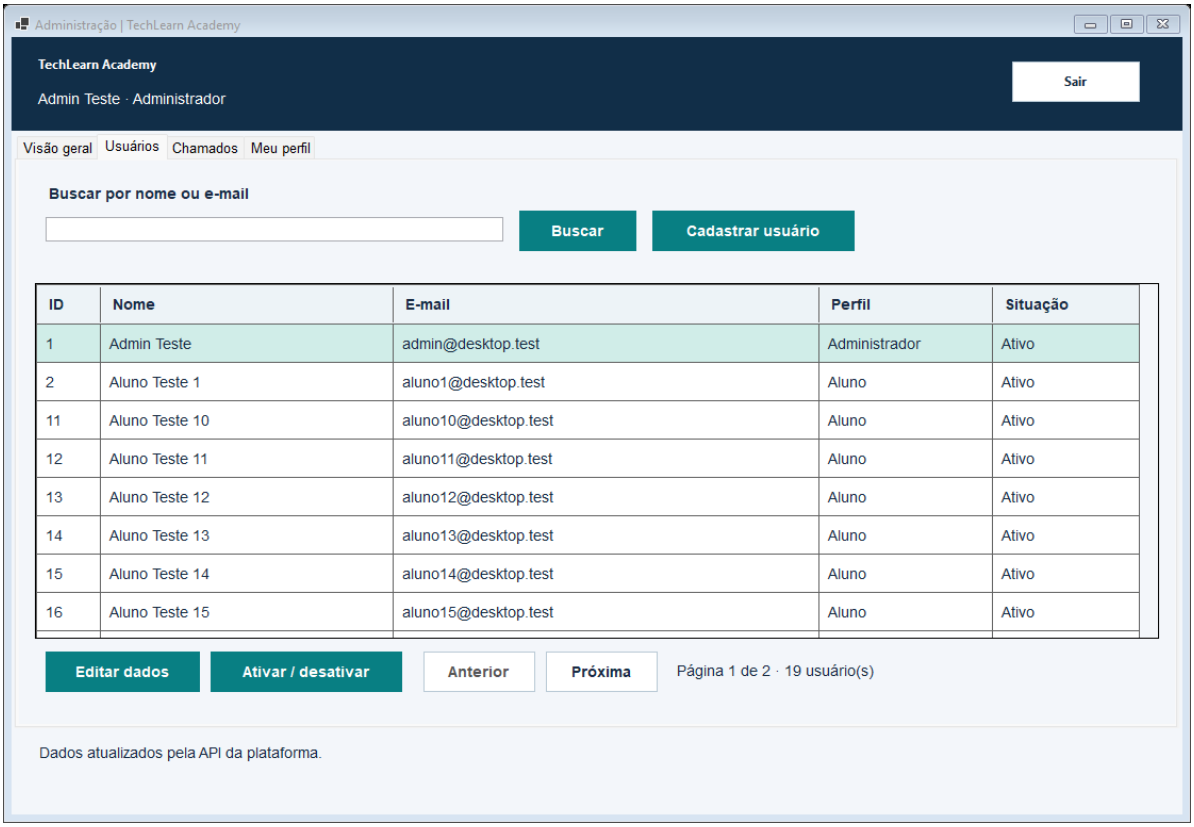
Encapsulamento aparece em Usuario, com private set e métodos de atualização e senha; em Matricula.Reiniciar; e em Resultado, que calcula nota, percentual e aprovação. Questao valida duas a cinco alternativas e uma correta. Usuario.Perfil representa os quatro papéis, sem subclasses por perfil.

Controllers herdam de BaseController; SessaoHandler sobrescreve a autenticação de AuthenticationHandler, evidenciando herança e polimorfismo na infraestrutura. As associações de negócio são frequentemente persistidas por IDs e JOIN. As classes centrais são sealed; organização em camadas e modularização não exigem herança entre elas.

7.2 Aplicação desktop administrativa

O desktop nativo Windows Forms usa C# e .NET 10 Windows para o Administrador. A interface usa TextBox, Button, TabControl e DataGridView. ApiCliente chama a API por HTTPS/JSON, com endereço em servidor.txt; não acessa SQL diretamente. A API valida sessão e perfil. O pacote contém executável e configuração para uso sem Visual Studio ou SQL Server na máquina final.

Figura 5 - Gestão de usuários no desktop nativo



Fonte: evidências do projeto desktop em ambiente de teste.

O C# em TechLearnAcademy.Desktop.Recuperado foi reconstruído por descompilação do EXE Windows Forms 1.0.2 e está disponível no Git. Contém TelaLogin, ApiCliente, TelaPrincipal, TelaUsuario e TelaChamado; o registro de 27/10 informa compilação sem erros. Não é o fonte original; faltam comparar o binário recompilado com o distribuído e registrar o aceite. O WebView2 antigo está no histórico.

O Administrador consulta e edita contas, cria Mentor e Gestor e ativa ou desativa usuários. O perfil não muda; a API protege o último Administrador ativo. Chamados do Gestor podem ser assumidos, respondidos e encerrados. A área em uso atualiza-se a cada cinco segundos e preserva o texto em digitação.

7.3 Correspondência entre código e comportamento

A orientação a objetos é observada quando os serviços criam e utilizam os modelos, e não apenas pela existência de arquivos de classe. No cadastro, UsuariosServico cria Usuario, solicita AtualizarDados, AtualizarDadosPessoais e DefinirSenha e só depois persiste os valores. Na correção, AvaliacoesServico cria Resultado e utiliza suas propriedades para gravar a tentativa.

Quadro 12 - Correspondência entre conceito e implementação

Conceito	Trecho ou membro real	Efeito no fluxo
Encapsulamento	Usuario.AtualizarDados e DefinirSenha	Controlam alterações e produção do hash.
Estado e comportamento	Matricula.Reiniciar(reprovado)	Incrementa a tentativa e zera progresso após regra válida.
Cálculo centralizado	new Resultado(acertos, total)	Produz Percentual, Nota e Aprovado usados na persistência.
Herança e sobrescrita	BaseController; SessaoHandler.HandleAuthenticateAsync	Reaproveitam o contrato do framework e especializam a autenticação.
Integração em camadas	Controller → serviço → ConexaoBanco	Separa entrada HTTP, regra e execução SQL.

Fonte: elaboração dos autores (2026).

Em AvaliacoesServico.cs, linhas 167 a 171, o fluxo usa os valores calculados pelo objeto:

```
var resultado = new Modelos.Resultado(acertos, questoes.Count);
int total = resultado.Total;
decimal percentual = resultado.Percentual;
decimal nota = resultado.Nota;
bool aprovado = resultado.Aprovado;
```

Curso não possui Id; Modulo não conserva CursoId como propriedade; Material não contém os bytes do PDF. Esses dados são tratados no pedido, no serviço e no SQL. Resultado é um objeto de cálculo persistido em TentativasAvaliacao. O modelo de classes não deve ser confundido com o esquema completo do banco.

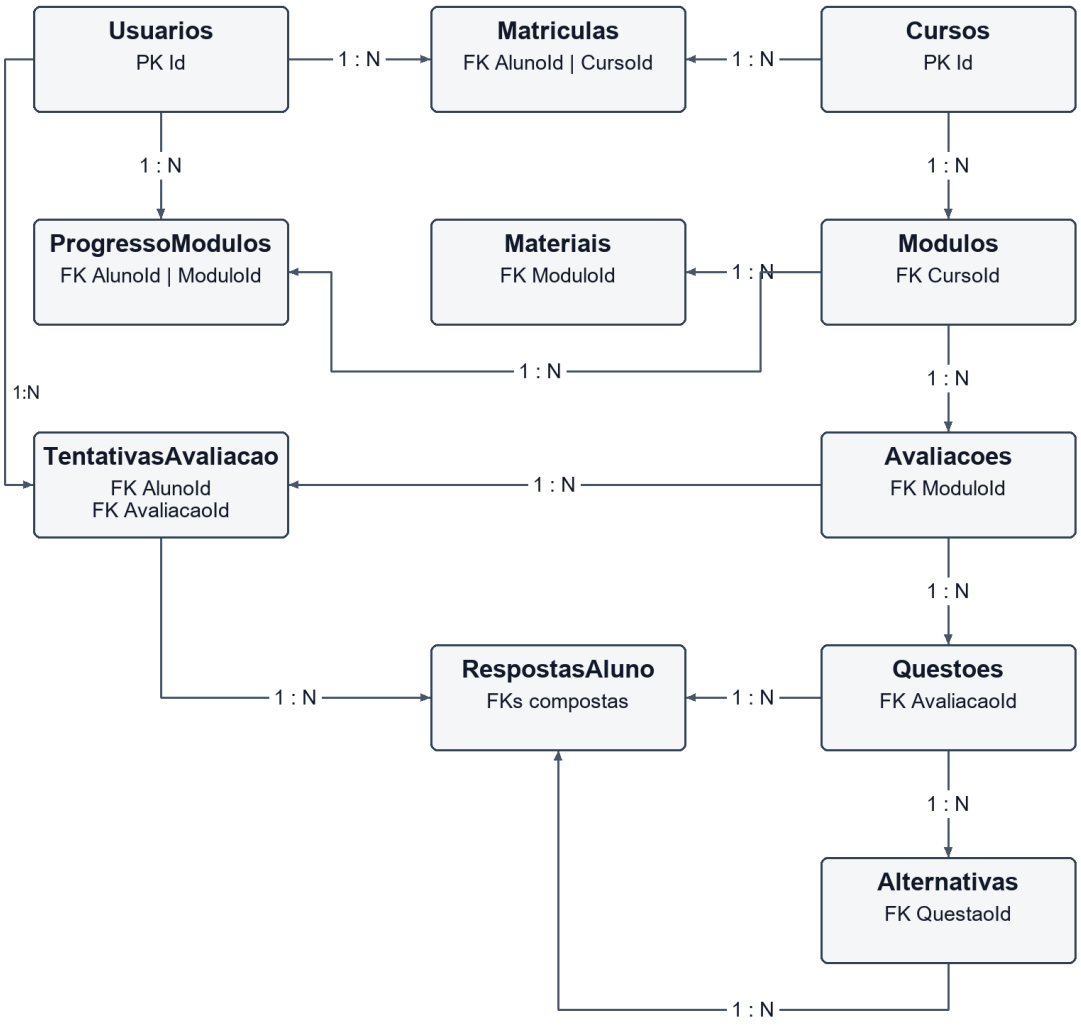
Para a banca, usar Diagramas/TechLearnAcademy-Diagramas-simples.asta, editável no Astah, e as 44 PNGs em Diagramas/Imagens. Há 14 casos de uso e quatro tipos de UML. Certificados e relatórios não têm caso próprio nessa versão. O fluxo real de avaliação continua na API; o Apêndice B relaciona modelagem e código.

8 PROJETO DO BANCO DE DADOS

8.1 Modelo entidade-relacionamento acadêmico

O modelo representa 19 tabelas em três pranchas complementares. Heuser (s.d.) situa a modelagem conceitual e o projeto lógico no ciclo de projeto de dados. As figuras usam os nomes físicos para facilitar a comparação com SQL e dicionário. PK identifica chave primária; FK, chave estrangeira; 1:N indica cardinalidade máxima.

Figura 6 - MER do núcleo acadêmico com onze entidades

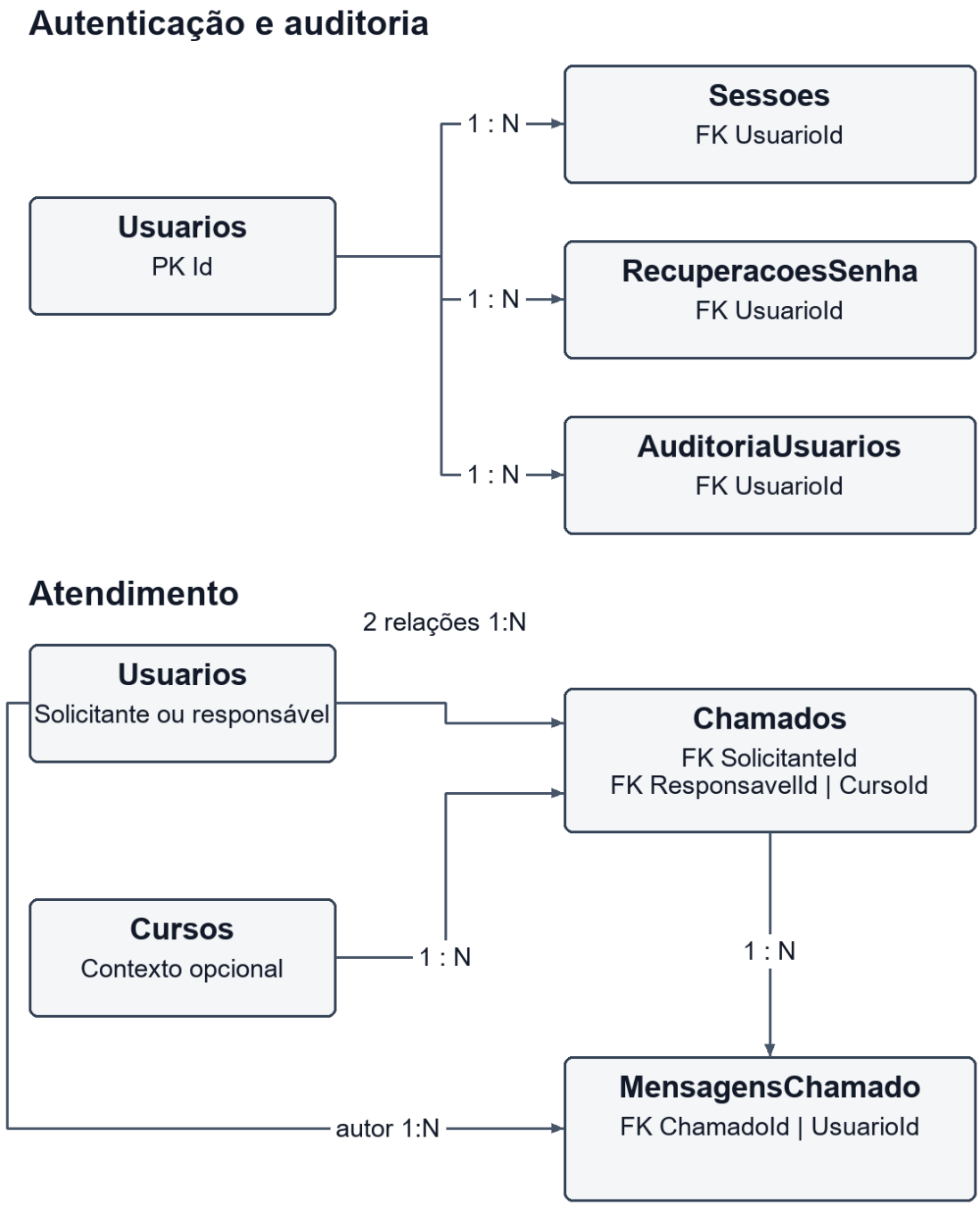


Fonte: elaboração dos autores (2026), com base nos scripts do projeto.

Matrículas relacionam alunos e cursos; módulos reúnem materiais e avaliações. Tentativas e respostas preservam provas; ProgressoModulos registra a conclusão. As FKs compostas de RespostasAluno conferem tentativa, avaliação, questão e alternativa no mesmo contexto.

8.2 Modelo de autenticação e atendimento

Figura 7 - MER de sessões, recuperação, auditoria e chamados

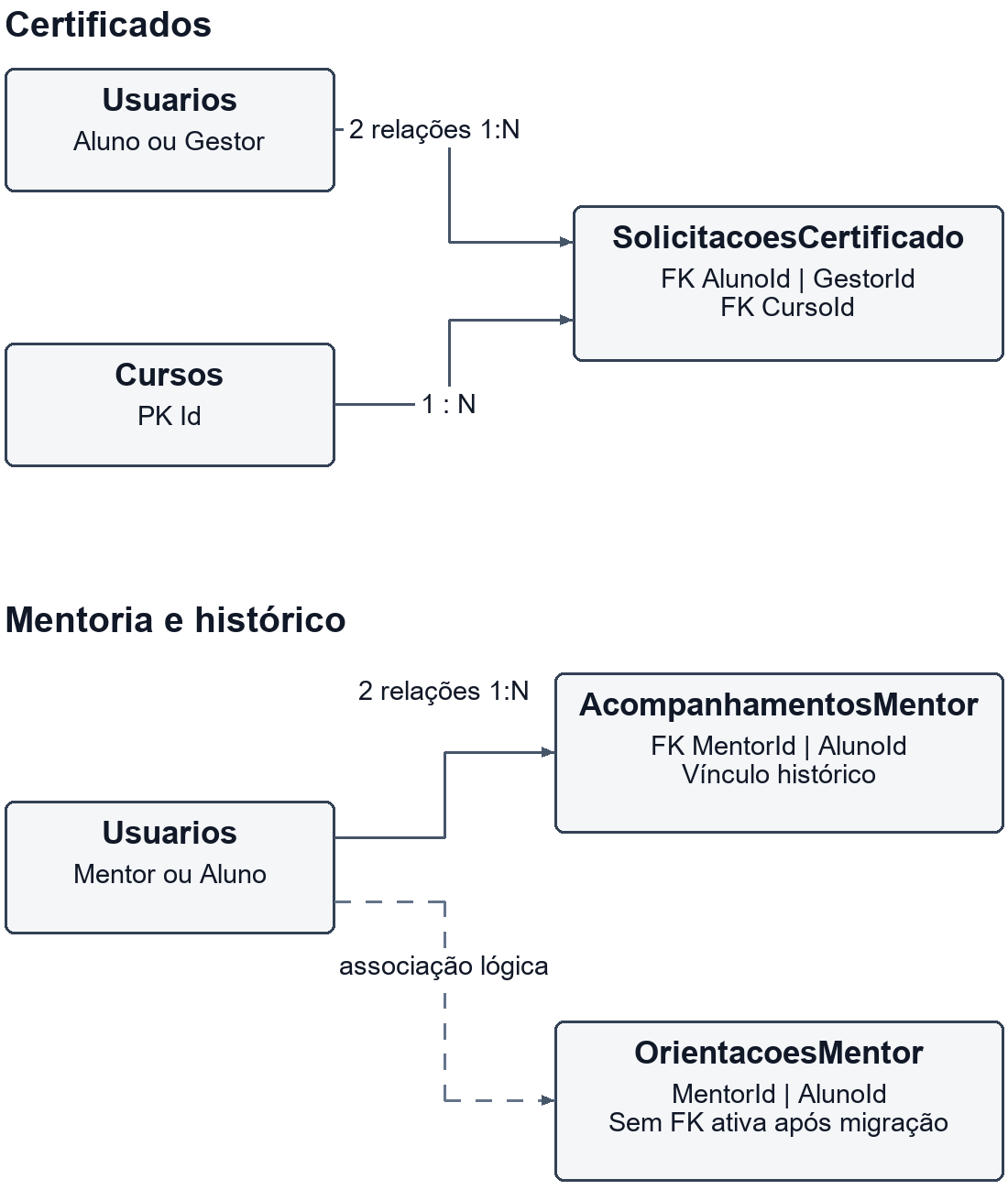


Fonte: elaboração dos autores (2026), com base nos scripts do projeto.

As cinco tabelas adicionais são Sessoes, RecuperacoesSenha, AuditoriaUsuarios, Chamados e MensagensChamado. Usuarios e Cursos reaparecem apenas como contexto. ResponsavelId e CursoId do chamado são opcionais; cada mensagem identifica o chamado e seu autor. Sessão e recuperação mantêm tokens em hash com validade.

8.3 Modelo de certificados e mentoria

Figura 8 - MER de certificados e das estruturas de mentoria



Fonte: elaboração dos autores (2026), com base nos scripts do projeto.

SolicitacoesCertificado relaciona aluno, curso e gestor; GestorId admite nulo antes da decisão. AcompanhamentosMentor conserva o vínculo histórico. OrientacoesMentor mantém os identificadores de mentor e aluno, mas a atualização remove a FK composta antiga; a linha tracejada indica uma associação lógica validada pela aplicação. As três pranchas cobrem todas as 19 tabelas, sem agrupá-las em uma única entidade.

8.4 Modelo lógico e dicionário do núcleo acadêmico

O modelo lógico usa Id como PK, FKs explícitas, textos nvarchar, notas decimais e datas datetime2 em UTC. Em Usuarios, Telefone nvarchar(11), Sexo nvarchar(20) e SexoOutro nvarchar(60) aceitam nulo para contas antigas. O cadastro novo exige os dados; CHECKs no SQL limitam dígitos, opções de gênero e descrição quando Sexo é Outro.

Quadro 13 - Dicionário resumido de usuários e conteúdos

Relação	Principais atributos e regras
Usuarios	Id PK; Nome; Email único; EmailContato opcional/único; SenhaHash; Perfil; Ativo; Telefone nvarchar(11) NULL (10 ou 11 dígitos); Sexo nvarchar(20) NULL (Masculino, Feminino ou Outro); SexoOutro nvarchar(60) NULL (apenas para Outro, 2 a 60 caracteres). Três CHECKs.
Cursos	Id PK; Nome nvarchar(150); Descricao nvarchar(2000); CargaHoraria positiva; Ativo; Excluido.
Modulos	Id PK; CursoId FK; Titulo; Ordem positiva; Ativo; Excluido; ordem única por curso não excluído.
Materiais	Id PK; ModuloId FK; Titulo; Tipo PDF ou Link; Url; Ativo; Excluido; ArquivoNome; ArquivoPdf varbinary(max).
Matriculas	Id PK; AlunoId e CursoId FKs; Ativo; DataMatricula; TentativaCurso positiva; uma matrícula ativa por par.
Avaliacoes	Id PK; ModuloId FK; Titulo; Ativo; Excluido.
Questoes	Id PK; AvaliacaoId FK; Enunciado nvarchar(2000); Ativo; chave única composta Id e AvaliacaoId.
Alternativas	Id PK; QuestaoId FK; Texto nvarchar(1000); Correta bit; no máximo uma correta por questão no índice filtrado.

Fonte: elaboração dos autores (2026).

O índice de alternativa correta assegura no máximo uma marcação por questão. A existência de exatamente uma correta e a quantidade de duas a cinco alternativas dependem também da validação de Questao na API. Essa distinção explicita o que o banco garante isoladamente e o que depende de uma operação de negócio completa.

O modelo evita repetir nome de curso em cada matrícula e enunciado em cada resposta. A informação é recuperada por relacionamento. O certificado é uma exceção deliberada: nomes e carga horária são copiados no momento da emissão para preservar o documento histórico mesmo após alterações cadastrais.

8.5 Modelo lógico de resultados e suporte

Quadro 14 - Dicionário resumido de resultados e operações

Relação	Atributos e função
TentativasAvaliacao	AlunoId, AvaliacaoId, Acertos, TotalQuestoes, Percentual decimal(5,2), Nota decimal(4,2), Aprovado, DataTentativa e TentativaCurso.
RespostasAluno	TentativaId, AvaliacaoId, QuestaoId, AlternativaId e Correta; unicidade de questão por tentativa e FKs compostas.
ProgressoModulos	AlunoId, ModuloId e DataConclusao; par aluno e módulo único.
Sessoes	UsuarioId, TokenHash char(64), Validade, Revogada e DataCriacao.
RecuperacoesSenha	UsuarioId, TokenHash char(64), Validade, Utilizado e DataCriacao.
Chamados	SolicitanteId, ResponsavelId opcional, CursoId opcional, Tipo, DestinatarioPerfil, Assunto, Descricao, Status e datas.
MensagensChamado	ChamadoId, UsuarioId, Texto nvarchar(4000) e DataEnvio.
SolicitacoesCertificado	AlunoId, CursoId, Status, GestorId, Motivo, Codigo e dados congelados de emissão; unicidade aluno e curso.
OrientacoesMentor	MentorId, AlunoId, Texto nvarchar(4000) e DataOrientacao.
AcompanhamentosMentor	MentorId, AlunoId e Ativo; tabela preservada por compatibilidade histórica.
AuditoriaUsuarios	UsuarioId, perfis e situações anterior e nova e DataAlteracao.

Fonte: elaboração dos autores (2026).

Todas as relações listadas possuem Id como chave primária. Campos de identidade, vínculo e conteúdo essencial são obrigatórios; campos associados a etapas futuras do processo podem ser nulos. Um chamado ainda não assumido tem ResponsavelId nulo; um certificado pendente ainda não possui código de emissão. O script integral explicita nulabilidade, valores padrão e cada restrição.

As FKs compostas de RespostasAluno conferem conjuntamente a tentativa e a avaliação, a questão e a avaliação, e a alternativa e a questão. Assim, a mera existência dos identificadores não permite registrar uma alternativa pertencente a outra prova. O campo AvaliacaoId nessa relação auxilia a integridade contextual.

8.6 Procedures, triggers e transações

Stored procedures encapsulam comandos executados no banco e podem receber parâmetros para operações recorrentes (MICROSOFT, s.d.c). O projeto contém sp_RealizarMatricula e sp_RelatorioCurso. A primeira valida aluno e curso ativos, confere duplicidade e insere a matrícula em transação. Bloqueios UPDLOCK e HOLDLOCK ajudam a serializar requisições concorrentes para o mesmo contexto.

sp_RelatorioCurso agrega matrículas, média de notas, aprovação e progresso. A versão do script considera o conjunto de tentativas históricas em parte dessas métricas. O serviço atual também produz relatórios específicos por tentativa do curso. Esses conjuntos não devem ser tratados como equivalentes sem observar os filtros: o relatório acadêmico precisa distinguir histórico completo de desempenho válido na tentativa atual.

Quadro 15 - Objetos programáveis do banco

Objeto	Finalidade	Limite da garantia
sp_RealizarMatricula	Validar aluno e curso e criar vínculo.	Não substitui autorização HTTP do usuário autenticado.
sp_RelatorioCurso	Agregar indicadores por curso.	Exige leitura do período e das tentativas incluídas.
tr_AuditoriaUsuarios	Registrar alterações de perfil ou situação.	Não registra toda alteração de nome, contato ou autoria da ação.
tr_LimiteModulos	Impedir inserção acima de quatro módulos não excluídos.	É AFTER INSERT; edição e demais caminhos também exigem regra na API.

Fonte: elaboração dos autores (2026).

A correção é coordenada por AvaliacoesServico: valida a matrícula e o conjunto de respostas, verifica a tentativa atual, calcula o resultado e grava tentativa e respostas no mesmo contexto transacional. Uma falha não deve deixar metade da prova registrada. Resultado calcula percentual como 100 × acertos / total e nota como 10 × acertos / total, com duas casas decimais.

A aprovação exige pelo menos 60%. Cada avaliação é enviada uma vez por tentativa do curso. Após reprovação, o reinício incrementa TentativaCurso e zera o progresso, preservando notas anteriores. Testes de concorrência e de envio por aba antiga são relevantes porque uma validação feita apenas na interface não protege requisições simultâneas.

8.7 Modelo físico, otimização e script completo

O modelo físico materializa tabelas, restrições, índices e objetos programáveis em T-SQL. São utilizados índices para perfis e situação de usuário, catálogo ativo, matrículas por curso, avaliações por módulo, tentativas por aluno e filas de chamados. Índices filtrados preservam unicidade de matrícula ativa, de alternativa correta e de códigos de certificado não nulos.

A otimização proposta começa por consultas reais e planos de execução, sem assumir ganho apenas pela existência de índices. Paginação limita as listas transferidas; filtros por contexto reduzem leituras desnecessárias. O armazenamento de PDF em varbinary(max) atende ao recorte atual, mas pode aumentar banco, backup e transferência. Uma operação maior deve comparar esse custo com armazenamento de objetos protegido, mantendo a autorização pela API.

Quadro 16 - Documentação física e complementos eletrônicos

Artefato	Uso na entrega
Apêndice A	Script integral de instalação do banco, incluindo procedures, triggers e atualizações incorporadas.
Banco/Instalar-banco-do-site.sql	Cópia executável do script examinado, sem usuários ou senhas de demonstração.
Banco/07-corrigir-recuperacao-e-avaliacoes.sql	Atualização de banco existente para tentativa de curso e contatos únicos.
dicionario_fisico.md	Extração de todas as definições de colunas e das alterações no script.
inventario_api.md	Relação de rotas e autorização extraída dos controllers atuais.

Fonte: elaboração dos autores (2026).

Uma instalação nova deve utilizar instância de teste e o script completo. Ele cria TechLearnAcademy_PIMIV e recusa a criação se o banco já existir. O script local contém CREATE DATABASE, USE e GO e não deve ser enviado diretamente como migração de Azure SQL Database. Nessa modalidade, o banco é provisionado antes e o esquema precisa ser aplicado com adaptações ao ambiente e validação em homologação.

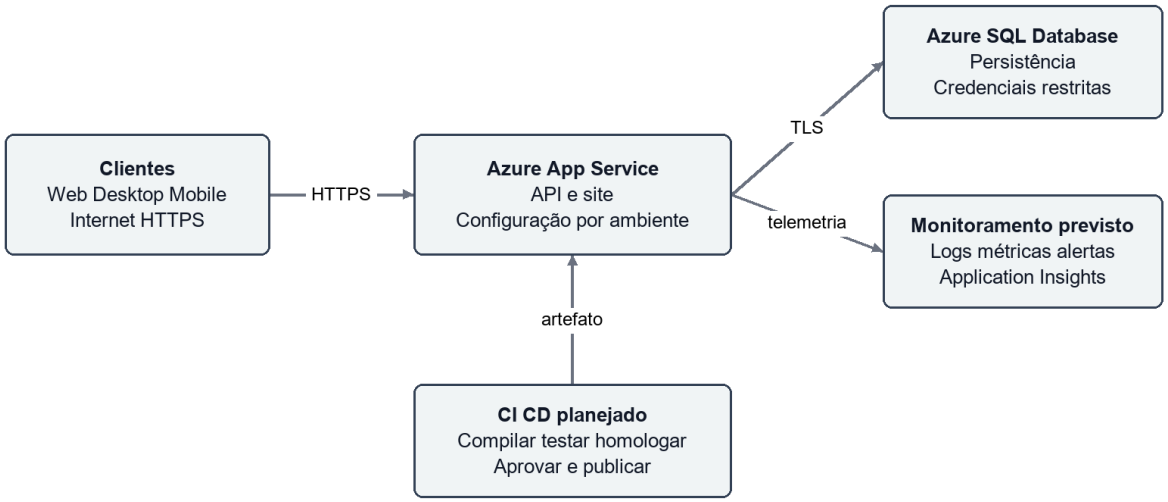
Os scripts de manutenção exigem caminhos reais e restauração em destino separado. A criação em ambiente vazio e os testes das restrições devem confirmar a integridade antes de qualquer implantação.

9 INFRAESTRUTURA EM NUVEM E DEVOPS

9.1 Arquitetura, serviços e segurança operacional

Web e API estão no Azure App Service com Azure SQL; o desktop chama a API por HTTPS. O registro de 07/11/2026 relata 174 arquivos publicados conferidos por hash, API compilada em Release sem erros ou avisos e importação pontual para LocalDB. A cópia tinha 10 usuários, 8 cursos, 27 módulos, 52 materiais, 7 matrículas, 4 chamados e 14 tentativas. São resultados relatados naquele registro, não reexecução nesta revisão; a cópia local não sincroniza automaticamente. App Service hospeda aplicações web e APIs (MICROSOFT, s.d.b).

Figura 9 - Arquitetura de nuvem e evolução operacional



Fonte: elaboração dos autores (2026).

O plano propõe separar desenvolvimento, homologação e produção por configuração, credenciais e base de dados. ConnectionStrings e parâmetros de e-mail devem ser fornecidos pelo ambiente; segredos não devem acompanhar o repositório ou o executável. A conta da aplicação no banco deve receber apenas as permissões necessárias, e a comunicação deve usar TLS com validação de certificado.

O endereço publicado não comprova rede privada, cofre de segredos ou redundância. O plano prevê restringir o banco, validar recuperação e configurar alertas. A escala depende de latência, erros e consumo de recursos; sessões compartilhadas exigem teste de concorrência. Recursos, região e custos devem ser conferidos na conta da organização.

9.2 Containers, pipeline, monitoramento e recuperação

O repositório contém Dockerfile com duas etapas: compilação pela imagem SDK .NET 10 e execução pela imagem ASP.NET 10. O contêiner publica a API e os arquivos web, utiliza usuário app e escuta na porta interna 8080. O término de HTTPS e o encaminhamento de cabeçalhos devem ser configurados no serviço de hospedagem. O arquivo comprova uma opção de empacotamento; não comprova que o site publicado utilize esse contêiner.

Quadro 17 - Plano de integração e entrega contínuas

Etapa do pipeline proposto	Verificação ou saída
Receber alteração	Revisar escopo, dependências e mudanças de esquema.
Compilar	Gerar API, arquivos web e cliente desktop; validar alvos mobile.
Testar	Executar regras, integração em SQL isolado e verificações de JavaScript.
Empacotar	Gerar artefato versionado ou imagem de contêiner.
Homologar	Aplicar esquema revisado e executar jornada com contas de teste.
Aprovar e publicar	Responsável autoriza promoção da versão validada.
Observar e recuperar	Acompanhar métricas e reverter artefato se houver regressão.

Fonte: elaboração dos autores (2026).

O pipeline é um plano de execução, não um workflow já demonstrado no repositório. A mudança de esquema deve ter cópia de segurança, validação de compatibilidade e revisão humana. Reverter o binário não desfaz automaticamente uma migração de dados; por isso, a estratégia deve definir versões compatíveis e forma de restauração antes de publicar.

O monitoramento proposto acompanha disponibilidade HTTP, tempo de resposta, erros 5xx, falhas de login, conflitos de matrícula, duração de consultas e crescimento do banco. Alertas devem indicar ação e responsável, evitando registrar senha, token, conexão completa ou conteúdo sensível de chamados.

Como meta inicial de planejamento, propõem-se RPO de 24 horas e RTO de quatro horas, sujeitos à validação do ambiente. Backups devem ser testados por restauração em destino diferente. Azure SQL requer política de recuperação própria do serviço; scripts de arquivo .bak do ambiente local não equivalem à política de backup da nuvem. Ensaios de falha, restauração e acesso móvel compõem a preparação operacional.

10 GERENCIAMENTO ÁGIL E VALIDAÇÃO

10.1 Product Backlog e priorização

O planejamento combina Scrum e visualização Kanban. Product Backlog, Sprint Backlog e definição de concluído organizam valor e aceite (SCHWABER; SUTHERLAND, 2020). Os quadros preservam a evolução do escopo e incorporam a entrega Android e os registros de usabilidade. Situação documental não equivale à execução de todos os testes.

Quadro 18 - Backlog consolidado e evolução do produto

ID	Entrega e aceite resumido	Prioridade
PB01	Cadastro de Aluno com e-mail único e dados válidos.	Alta
PB02	Login, sessão e separação por perfil no servidor.	Alta
PB03	Painel web e gestão de conteúdo autorizada.	Alta
PB04	Jornada Android integrada, com homologação em dispositivo.	Alta
PB05	Curso criado, consultado e editado pelo Gestor.	Alta
PB06	Módulo relacionado ao curso e limite respeitado.	Alta
PB07	PDF ou link acessível ao Aluno matriculado.	Alta
PB08	Matrícula confirmada sem duplicação ativa.	Alta
PB09	Avaliação vinculada ao módulo correto.	Alta
PB10	Questões com duas a cinco alternativas e uma correta.	Alta
PB11	Envio completo registrado uma vez por tentativa.	Alta
PB12	Correção no servidor conforme gabarito.	Alta
PB13	Nota, acertos e percentual correspondentes.	Alta
PB14	Progresso obtido por aprovação das avaliações.	Alta
PB15	Interfaces avaliadas por critérios de acessibilidade.	Alta
PB16	Dado criado na web consultado por outro canal.	Alta
PB17	Chamado assumido e conversa restrita aos autorizados.	Evolução
PB18	Certificado emitido somente com aprovação elegível.	Evolução
PB19	Desktop administrativo usa API e sessão reais.	Evolução

Fonte: elaboração dos autores (2026).

PB01 a PB16 preservam os IDs iniciais com critérios atualizados. PB17 a PB19 são extensões. Autenticação precede conteúdo; matrícula precede avaliação e certificado. Mobile e acessibilidade exigem homologação.

10.2 Sprint Backlog de consolidação

A Sprint 4 consolida documentação, entrega e aceite. Luis atua como Product Owner; Raphael, Scrum Master; Fábio e José Guilherme, Dev Team. A distribuição abaixo mantém as responsabilidades de referência. “Concluído” identifica um artefato disponível ou uma execução histórica registrada; homologações sem relatório permanecem abertas.

Quadro 19 - Sprint Backlog detalhado da Sprint 4

ID	Tarefa	Responsável	Critério de aceite	Situação
S4-01	Documentar Android atual	Luis e Raphael	APK e capturas descritos no capítulo 6.	Concluído
S4-02	Homologar M01 a M06	Luis e Raphael	Relatório por aparelho, versão e caso.	A fazer
S4-03	Conferir MER e UML	José Guilherme	Diagramas comparados e aceite do grupo.	Em validação
S4-04	Conferir API publicada	Luis	Preservar evidência HTTP de 18/10.	Concluído
S4-05	Conciliar SQL e dicionário	José Guilherme	19 tabelas e atualizações identificadas.	Concluído
S4-06	Consolidar Product Backlog	Fábio	Escopo e extensões identificados.	Concluído
S4-07	Atualizar Kanban	Fábio	Mesmos IDs e situações da sprint.	Concluído
S4-08	Revisar texto e referências	Raphael	Sumário e paginação conferidos.	Concluído
S4-09	Avaliar acessibilidade	Raphael e Luis	Teclado, contraste e leitor de tela.	A fazer
S4-10	Aceitar entrega do conjunto	Todos	Artefatos e pendências conferidos.	Em validação
S4-11	Registrar usabilidade	Todos	Roteiro, três participantes e evidências.	Concluído
S4-12	Conferir fontes dos aplicativos	Todos	Código desktop reconstruído disponível; comparar binário e obter fontes Java.	Em validação

Fonte: elaboração dos autores (2026).

S4-02 segue aberta: faltam relatórios M01 a M06, apesar de duas capturas. S4-12 está em validação: o código desktop nativo foi recuperado e disponibilizado, mas faltam comparar o binário recompilado e obter o projeto Java do APK. S4-10 depende da conferência coletiva. Texto atualizado não substitui homologação.

10.3 Cronograma, Kanban e entregas

Quadro 20 - Cronograma de referência das sprints

Sprint	Período relativo	Entrega e dependências
S1	Semanas 1 e 2	Diagnóstico, organização, requisitos e modelo de negócio.
S2	Semanas 3 e 4	Escopo, backlog, esquema SQL e autenticação.
S3	Semanas 5 e 6	Conteúdo, matrícula, avaliação e resultados integrados.
S4	Semanas 7 e 8	Consolidação S4-01 a S4-12; aceite depende de testes, fontes e acessibilidade.

Fonte: elaboração dos autores (2026).

As semanas constituem um cronograma relativo de referência. Não foram atribuídas datas ou horas a reuniões sem registro. A revisão incorpora a documentação do APK e a atividade do Dia da UNIP como evidências posteriores; seus registros não demonstram, por si sós, a execução de toda a Sprint 4.

Quadro 21 - Kanban consolidado das tarefas de revisão

A fazer	Bloqueado	Em validação	Concluído
S4-02 Homologar Android	Sem impedimento formal registrado.	S4-03 Conferir diagramas	S4-01 e S4-04 Mobile e registro HTTP
S4-09 Acessibilidade		S4-10 Aceite do conjunto	S4-05 e S4-06 SQL e backlog
		S4-12 Fontes e binários	S4-07 e S4-08 Kanban e texto
			S4-11 Registro de usabilidade

Fonte: elaboração dos autores (2026).

Os cartões mantêm os IDs do Sprint Backlog. O limite proposto é de dois itens simultâneos em execução ou validação. Concluído significa que existe uma saída identificável no acervo; homologação exige o resultado do ensaio e aceite. A coluna Bloqueado não recebe motivo presumido quando o impedimento não foi formalmente registrado.

10.4 Evidências, testes e rastreabilidade

Os relatórios históricos citam 189 registros: 186 em quatro arquivos e três consultas HTTP datadas. A soma conta registros, não cenários únicos nem cobertura percentual. Os arquivos JSON dessa série não estão versionados no ramo main conferido em 08/11/2026; a árvore contém os scripts de teste, mas a presença dos scripts não comprova nova execução. A usabilidade com três participantes é apresentada separadamente.

Quadro 22 - Inventário único dos registros de verificação

Arquivo histórico citado	Qtd.	Abrangência
integracao.json	30	API, autenticação, curso, avaliação e persistência.
adicionais.json	17	Regras, concorrência, integridade e erros.
testes.json	54	Desktop, interface, sessão e integração com API.
colagem.json	85	Controles nativos, atalhos e preservação de texto.
azure_revisao.json	3	Página web, disponibilidade da API e proteção do perfil.
TOTAL	189	Registros citados, ainda sem arquivos JSON na árvore Git atual.

Fonte: elaboração dos autores (2026).

Quadro 23 - Conferência do endereço publicado

Requisição sem credenciais	Resposta em 18 de outubro de 2026
GET /	200; página de autenticação entregue.
GET /api/status	200; API disponível.
GET /api/usuarios/perfil	401; acesso protegido recusado sem login.

Fonte: elaboração dos autores (2026).

Endereço público: https://techlearn-proj-pim-hyayf7cehyd3edfq.westus3-01.azurewebsites.net/

Os relatórios históricos citam registros de teste, dicionário, inventário de API e matriz de cobertura. Parte das evidências JSON não consta da árvore Git de 08/11/2026; sua inclusão na entrega deve ser conferida pelo grupo. A publicação e a comparação registradas em 07/11 são evidências pontuais, sem garantia de disponibilidade contínua. O PDF versionado no Git recebeu uma atualização posterior à revisão textual; a entrega deve usar a exportação conferida deste Word. O Apêndice D distingue arquivo presente, resultado anterior e aceite pendente.

10.4.1 Teste unitário da confirmação de senha

Em 8 de novembro de 2026, foi acrescentado à solução o projeto TechLearnAcademy.UnitTests, configurado com xUnit e referenciado pela solução principal. O teste SenhasDiferentes_DevemSerRecusadas verifica diretamente a regra Regras.Senha, utilizada pelo cadastro da API. Seu objetivo é confirmar que duas senhas individualmente válidas, porém diferentes, interrompem o cadastro com erro de validação HTTP 400.

Código 1 - Teste unitário da confirmação de senha

```
[Fact]
public void SenhasDiferentes_DevemSerRecusadas()
{
    string senha = "SenhaSegura1!";
    string confirmarSenha = "OutraSenha2!";

    ErroAplicacao erro = Assert.Throws<ErroAplicacao>(
        () => Regras.Senha(senha, confirmarSenha));

    Assert.Equal(400, erro.Status);
    Assert.Equal("As senhas devem ser exatamente iguais.", erro.Message);
}
```

Fonte: elaboração dos autores (2026).

O teste segue a organização Preparação, Ação e Verificação: prepara os valores, executa a regra real e compara o erro obtido com o comportamento esperado. Por testar somente a regra de senha, ele não abre o navegador, não acessa o SQL Server e não cria usuários. O código está versionado no Git; nesta revisão documental, sua execução no Gerenciador de Testes do Visual Studio e a captura do resultado permanecem pendentes.

10.5 Testes de usabilidade no Dia da UNIP

No laboratório de informática da UNIP Campus Dutra, três visitantes utilizaram o perfil de Aluno em um teste exploratório. O registro da atividade descreve o mesmo roteiro para todos: abrir chamado ao mentor, realizar avaliação e consultar o resultado e editar um dado permitido do perfil. Os integrantes acompanharam o uso e esclareceram dúvidas sem executar as ações no lugar dos participantes.

Quadro 24 - Síntese dos registros de usabilidade

Tarefa	Resultado descrito no relato	Aperfeiçoamento proposto
Solicitar ajuda	Roteiro concluído; suporte centralizado percebido como útil.	Avaliar destaque do acesso à ajuda e da confirmação do envio.
Realizar avaliação	Respostas enviadas e resultado consultado; nota e acertos compreensíveis.	Padronizar o acesso à avaliação e o retorno ao curso.
Editar perfil	Alteração de um dado permitida e concluída.	Dar maior destaque à confirmação de salvamento.

Fonte: síntese dos registros da atividade extensionista e análise dos autores (2026).

O relato informa que os três participantes concluíram as tarefas. As impressões favoráveis se relacionam à organização, ao suporte no próprio sistema e à apresentação dos resultados. As sugestões gerais tratam da visibilidade de comandos e confirmações. As propostas do quadro são interpretações de melhoria; não representam alterações já implementadas ou falas individuais transcritas.

Não foram disponibilizadas medições de tempo, contagem de erros, escala padronizada ou registro de ajuda por tarefa. O roteiro orientava ações e houve acompanhamento dos integrantes; portanto, os resultados não demonstram navegação totalmente autônoma nem permitem generalização estatística. O teste não substitui avaliação com tecnologias assistivas. O Apêndice C reúne o procedimento e registros visuais disponíveis.

11 CONCLUSÃO

O trabalho documentou a integração dos processos educacionais da TechLearn Academy. A proposta de acesso gratuito busca reduzir barreiras financeiras à formação em tecnologia e dar continuidade ao projeto acadêmico. O planejamento relaciona cursos, matrícula, avaliação, suporte e acompanhamento aos recursos necessários; a gratuidade depende de custeio e capacidade operacional a validar.

A solução reúne web em HTML, CSS e JavaScript, API ASP.NET Core em C#, SQL Server, desktop Windows Forms e Android nativo em Java. A API centraliza identidade, permissões, regras acadêmicas e persistência. Modelos com validações, serviços, SQL parametrizado e transações sustentam o fluxo; a UML explicita objetivos, classes e sequência de execução.

Os registros de usabilidade descrevem três participantes realizando tarefas de ajuda, avaliação e perfil no Dia da UNIP. A experiência oferece indícios de compreensão das funções e aponta melhorias em comandos e confirmações. Seu alcance é exploratório, condicionado ao roteiro e ao acompanhamento; não constitui validação estatística ou certificação de acessibilidade.

O C# desktop reconstruído está no Git para estudo, sem prova de equivalência com o fonte original ou o novo binário. O APK Java e duas capturas foram examinados; faltam os fontes Java/Gradle originais e os resultados M01 a M06 em aparelho.

Responsabilidade social e diversidade foram vinculadas à curadoria de conteúdos e ao atendimento. Recursos como foco visível e VLibras contribuem para o uso inclusivo, mas exigem verificação de contraste, teclado e tecnologias assistivas. Nuvem e DevOps contam com publicação e plano operacional; pipeline, monitoramento e recuperação devem ser demonstrados conforme forem executados.

A solução integra as disciplinas e oferece uma base para a organização fictícia. A documentação distingue fontes reconstruídos, registros antigos e o teste xUnit implementado, cuja execução registrada permanece pendente. Permanecem pendentes a homologação Android, os fontes Java, a comparação do desktop recompilado, a validação econômica e de acessibilidade e a decisão sobre a necessidade de telefone e gênero.

REFERÊNCIAS

BRASIL. **Lei nº 12.288, de 20 de julho de 2010**. Institui o Estatuto da Igualdade Racial. Brasília, DF: Presidência da República, 2010. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2007-2010/2010/lei/l12288.htm. Acesso em: 18 out. 2026.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais. Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 18 out. 2026.

GERHARDT, Tatiana Engel; SILVEIRA, Denise Tolfo (org.). **Métodos de pesquisa**. Porto Alegre: Editora da UFRGS, 2009. Disponível em: https://www.idea.ufscar.br/arquivos/metodologia/delineamentos/pdf-gerhardt-e-silveira-2009-metodos-de-pesquisa.pdf. Acesso em: 18 out. 2026.

GOOGLE. Introduction to activities. Android Developers, [s.d.a]. Disponível em: https://developer.android.com/guide/components/activities/intro-activities. Acesso em: 1 nov. 2026.

GOOGLE. Processes and threads overview. Android Developers, [s.d.b]. Disponível em: https://developer.android.com/guide/components/processes-and-threads. Acesso em: 1 nov. 2026.

HEUSER, Carlos Alberto. Principais características. Projeto de Banco de Dados, [s.d.]. Disponível em: https://projbd.heuser.pro.br/caracteristicas/. Acesso em: 1 nov. 2026.

LAVERTY, Michael; LITTEL, Chris. **Entrepreneurship**. Houston: OpenStax, 2020. Seção 11.2 Designing the Business Model. Disponível em: https://openstax.org/books/entrepreneurship/pages/11-2-designing-the-business-model. Acesso em: 18 out. 2026.

MICROSOFT. **Overview of ASP.NET Core**. Microsoft Learn, [s.d.a]. Disponível em: https://learn.microsoft.com/en-us/aspnet/core/overview?view=aspnetcore-10.0. Acesso em: 18 out. 2026.

MICROSOFT. Overview of Azure App Service. Microsoft Learn, [s.d.b]. Disponível em: https://learn.microsoft.com/en-us/azure/app-service/overview. Acesso em: 18 out. 2026.

MICROSOFT. **Stored procedures Database Engine**. Microsoft Learn, [s.d.c]. Disponível em: https://learn.microsoft.com/en-us/sql/relational-databases/stored-procedures/stored-procedures-database-engine. Acesso em: 18 out. 2026.

OBJECT MANAGEMENT GROUP. Unified Modeling Language version 2.5.1. 2017. Disponível em: https://www.omg.org/spec/UML/2.5.1/About-UML/. Acesso em: 1 nov. 2026.

SCHWABER, Ken; SUTHERLAND, Jeff. **The Scrum Guide**. [S.l.]: Scrum Guides, 2020. Disponível em: https://scrumguides.org/scrum-guide.html. Acesso em: 18 out. 2026.

VALENTE, Marco Tulio. **Engenharia de software moderna**: princípios e práticas para desenvolvimento de software com produtividade. [S.l.]: edição independente, 2020. Capítulo 7. Disponível em: https://engsoftmoderna.info/cap7.html. Acesso em: 18 out. 2026.

WORLD WIDE WEB CONSORTIUM W3C. **Web Content Accessibility Guidelines WCAG 2.2**. W3C Recommendation, 12 dez. 2024. Disponível em: https://www.w3.org/TR/WCAG22/. Acesso em: 18 out. 2026.

APÊNDICE A SCRIPT COMPLETO DO BANCO DE DADOS

O código abaixo reproduz integralmente Banco/Instalar-banco-do-site.sql para instalação nova. A comparação com o blob Git d76a0a51e4c1dc3231226f170867c2e168f896c9 confirmou o mesmo conteúdo, incluindo 19 tabelas, duas procedures, duas triggers e Telefone, Sexo e SexoOutro em Usuarios. O SQL é a versão de 08/11/2026 e não foi executado nesta revisão.

O script cria TechLearnAcademy_PIMIV somente em instância SQL Server de teste sem esse banco e recusa a operação caso ele já exista. Para uma base existente, o repositório fornece Banco/08-dados-pessoais-usuarios.sql: ele adiciona as três colunas nulas e os CHECKs sem apagar os usuários. Antes de aplicá-lo, conferir conexão e backup. A instalação local com CREATE DATABASE, USE e GO não deve ser enviada diretamente como migração do Azure SQL.

```
-- Execute este arquivo uma única vez no SSMS conectado a (localdb)\MSSQLLocalDB.
-- Ele cria um banco separado para o site e mantém o banco TechLearnAcademy existente intacto.

USE master;
GO

IF DB_ID(N'TechLearnAcademy_PIMIV') IS NOT NULL
   THROW 51000, 'Já existe TechLearnAcademy_PIMIV nesta instância. Use uma instância isolada.', 1;

CREATE DATABASE TechLearnAcademy_PIMIV COLLATE Latin1_General_100_CI_AS_SC;
GO

-- Mantém o banco aberto entre os acessos do site.
ALTER DATABASE TechLearnAcademy_PIMIV SET AUTO_CLOSE OFF;
GO

USE TechLearnAcademy_PIMIV;
GO

-- Índices filtrados exigem estas opções em todas as conexões.
SET ANSI_NULLS ON;
SET QUOTED_IDENTIFIER ON;
SET ANSI_PADDING ON;
SET ANSI_WARNINGS ON;
SET ARITHABORT ON;
SET CONCAT_NULL_YIELDS_NULL ON;
SET NUMERIC_ROUNDABORT OFF;

-- Se um objeto falhar, a conexão desfeita não deixa um esquema parcial.
BEGIN TRANSACTION;

-- Guarda somente o hash da senha, nunca a senha digitada.
CREATE TABLE dbo.Usuarios (
   Id int IDENTITY PRIMARY KEY,
   Nome nvarchar(100) NOT NULL,
   Email nvarchar(254) NOT NULL,
   Telefone nvarchar(11) NULL,
   Sexo nvarchar(20) NULL,
   SexoOutro nvarchar(60) NULL,
   SenhaHash nvarchar(500) NOT NULL,
   Perfil nvarchar(20) NOT NULL DEFAULT N'Aluno',
   Ativo bit NOT NULL DEFAULT 1,
   DataCadastro datetime2 NOT NULL DEFAULT SYSUTCDATETIME(),
   CONSTRAINT UQ_Usuarios_Email UNIQUE (Email),
   CONSTRAINT CK_Usuarios_Nome CHECK (LEN(LTRIM(RTRIM(Nome))) BETWEEN 2 AND 100),
```

```
    CONSTRAINT CK_Usuarios_Email CHECK (LEN(Email) BETWEEN 3 AND 254 AND Email = LTRIM(RTRIM(Email))),
    CONSTRAINT CK_Usuarios_Telefone CHECK (Telefone IS NULL OR (LEN(Telefone) IN (10,11) AND Telefone NOT LIKE
'%[^0-9]%')),
    CONSTRAINT CK_Usuarios_Sexo CHECK (Sexo IS NULL OR Sexo IN (N'Masculino',N'Feminino',N'Outro')),
    CONSTRAINT CK_Usuarios_SexoOutro CHECK ((ISNULL(Sexo,N'')=N'Outro' AND SexoOutro IS NOT NULL AND
LEN(LTRIM(RTRIM(SexoOutro))) BETWEEN 2 AND 60) OR (ISNULL(Sexo,N'')<>N'Outro' AND SexoOutro IS NULL)),
    CONSTRAINT CK_Usuarios_Hash CHECK (LEN(SenhaHash) >= 20),
    CONSTRAINT CK_Usuarios_Perfil CHECK (Perfil IN (N'Aluno', N'Mentor', N'Gestor', N'Administrador'))
);

CREATE TABLE dbo.RecuperacoesSenha (
    Id int IDENTITY PRIMARY KEY,
    UsuarioId int NOT NULL REFERENCES dbo.Usuarios(Id),
    TokenHash char(64) NOT NULL UNIQUE,
    Validade datetime2 NOT NULL,
    Utilizado bit NOT NULL DEFAULT 0,
    DataCriacao datetime2 NOT NULL DEFAULT SYSUTCDATETIME(),
    CONSTRAINT CK_RecuperacoesSenha_Validade CHECK (Validade > DataCriacao),
    CONSTRAINT CK_RecuperacoesSenha_Hash CHECK (TokenHash NOT LIKE '%[^0-9A-Fa-f]%' COLLATE
Latin1_General_100_BIN2)
);

CREATE TABLE dbo.Sessoes (
    Id int IDENTITY PRIMARY KEY,
    UsuarioId int NOT NULL REFERENCES dbo.Usuarios(Id),
    TokenHash char(64) NOT NULL UNIQUE,
    Validade datetime2 NOT NULL,
    Revogada bit NOT NULL DEFAULT 0,
    DataCriacao datetime2 NOT NULL DEFAULT SYSUTCDATETIME(),
    CONSTRAINT CK_Sessoes_Validade CHECK (Validade > DataCriacao),
    CONSTRAINT CK_Sessoes_Hash CHECK (TokenHash NOT LIKE '%[^0-9A-Fa-f]%' COLLATE Latin1_General_100_BIN2)
);

CREATE TABLE dbo.Cursos (
    Id int IDENTITY PRIMARY KEY,
    Nome nvarchar(150) NOT NULL,
    Descricao nvarchar(2000) NOT NULL,
    CargaHoraria int NOT NULL,
    Ativo bit NOT NULL DEFAULT 1,
    CONSTRAINT CK_Cursos_Nome CHECK (LEN(LTRIM(RTRIM(Nome))) BETWEEN 2 AND 150),
    CONSTRAINT CK_Cursos_Descricao CHECK (LEN(LTRIM(RTRIM(Descricao))) > 0),
    CONSTRAINT CK_Cursos_CargaHoraria CHECK (CargaHoraria > 0)
);

CREATE TABLE dbo.Modulos (
    Id int IDENTITY PRIMARY KEY,
    CursoId int NOT NULL REFERENCES dbo.Cursos(Id),
    Titulo nvarchar(150) NOT NULL,
    Ordem int NOT NULL,
    Ativo bit NOT NULL DEFAULT 1,
    CONSTRAINT UQ_Modulos_Ordem UNIQUE (CursoId, Ordem),
    CONSTRAINT CK_Modulos_Titulo CHECK (LEN(LTRIM(RTRIM(Titulo))) > 0),
    CONSTRAINT CK_Modulos_Ordem CHECK (Ordem > 0)
);

CREATE TABLE dbo.Materiais (
    Id int IDENTITY PRIMARY KEY,
    ModuloId int NOT NULL REFERENCES dbo.Modulos(Id),
    Titulo nvarchar(150) NOT NULL,
    Tipo nvarchar(10) NOT NULL,
    Url nvarchar(2000) NOT NULL,
    Ativo bit NOT NULL DEFAULT 1,
    CONSTRAINT CK_Materiais_Titulo CHECK (LEN(LTRIM(RTRIM(Titulo))) > 0),
    CONSTRAINT CK_Materiais_Tipo CHECK (Tipo IN (N'PDF', N'Link')),
```

```
    CONSTRAINT CK_Materiais_Url CHECK (Url LIKE N'https://%' OR Url LIKE N'http://%')
);

CREATE TABLE dbo.Matriculas (
    Id int IDENTITY PRIMARY KEY,
    AlunoId int NOT NULL REFERENCES dbo.Usuarios(Id),
    CursoId int NOT NULL REFERENCES dbo.Cursos(Id),
    Ativo bit NOT NULL DEFAULT 1,
    DataMatricula datetime2 NOT NULL DEFAULT SYSUTCDATETIME()
);

CREATE TABLE dbo.Avaliacoes (
    Id int IDENTITY PRIMARY KEY,
    ModuloId int NOT NULL REFERENCES dbo.Modulos(Id),
    Titulo nvarchar(150) NOT NULL,
    Ativo bit NOT NULL DEFAULT 1,
    CONSTRAINT CK_Avaliacoes_Titulo CHECK (LEN(LTRIM(RTRIM(Titulo))) > 0)
);

CREATE TABLE dbo.Questoes (
    Id int IDENTITY PRIMARY KEY,
    AvaliacaoId int NOT NULL REFERENCES dbo.Avaliacoes(Id),
    Enunciado nvarchar(2000) NOT NULL,
    Ativo bit NOT NULL DEFAULT 1,
    CONSTRAINT UQ_Questoes_Avaliacao UNIQUE (Id, AvaliacaoId),
    CONSTRAINT CK_Questoes_Enunciado CHECK (LEN(LTRIM(RTRIM(Enunciado))) > 0)
);

CREATE TABLE dbo.Alternativas (
    Id int IDENTITY PRIMARY KEY,
    QuestaoId int NOT NULL REFERENCES dbo.Questoes(Id),
    Texto nvarchar(1000) NOT NULL,
    Correta bit NOT NULL DEFAULT 0,
    CONSTRAINT UQ_Alternativas_Questao UNIQUE (Id, QuestaoId),
    CONSTRAINT CK_Alternativas_Texto CHECK (LEN(LTRIM(RTRIM(Texto))) > 0)
);

CREATE TABLE dbo.TentativasAvaliacao (
    Id int IDENTITY PRIMARY KEY,
    AlunoId int NOT NULL REFERENCES dbo.Usuarios(Id),
    AvaliacaoId int NOT NULL REFERENCES dbo.Avaliacoes(Id),
    Acertos int NOT NULL,
    TotalQuestoes int NOT NULL,
    Percentual decimal(5,2) NOT NULL,
    Nota decimal(4,2) NOT NULL,
    Aprovado bit NOT NULL,
    DataTentativa datetime2 NOT NULL DEFAULT SYSUTCDATETIME(),
    CONSTRAINT UQ_Tentativas_Avaliacao UNIQUE (Id, AvaliacaoId),
    CONSTRAINT CK_Tentativas_Acertos CHECK (TotalQuestoes > 0 AND Acertos BETWEEN 0 AND TotalQuestoes),
    CONSTRAINT CK_Tentativas_Percentual CHECK (Percentual BETWEEN 0 AND 100 AND Percentual = ROUND(100.0 *
Acertos / TotalQuestoes, 2)),
    CONSTRAINT CK_Tentativas_Nota CHECK (Nota BETWEEN 0 AND 10 AND Nota = ROUND(10.0 * Acertos /
TotalQuestoes, 2)),
    CONSTRAINT CK_Tentativas_Aprovacao CHECK ((Aprovado = 1 AND Percentual >= 60) OR (Aprovado = 0 AND
Percentual < 60))
);

-- As chaves compostas ligam tentativa, questão e alternativa corretamente.
CREATE TABLE dbo.RespostasAluno (
    Id int IDENTITY PRIMARY KEY,
    TentativaId int NOT NULL,
    AvaliacaoId int NOT NULL,
    QuestaoId int NOT NULL,
    AlternativaId int NOT NULL,
```

```
    Correta bit NOT NULL,
    CONSTRAINT UQ_Respostas_Questao UNIQUE (TentativaId, QuestaoId),
    CONSTRAINT FK_Respostas_Tentativa FOREIGN KEY (TentativaId, AvaliacaoId) REFERENCES
dbo.TentativasAvaliacao(Id, AvaliacaoId),
    CONSTRAINT FK_Respostas_Questao FOREIGN KEY (QuestaoId, AvaliacaoId) REFERENCES dbo.Questoes(Id,
AvaliacaoId),
    CONSTRAINT FK_Respostas_Alternativa FOREIGN KEY (AlternativaId, QuestaoId) REFERENCES dbo.Alternativas(Id,
QuestaoId)
);

CREATE TABLE dbo.ProgressoModulos (
    Id int IDENTITY PRIMARY KEY,
    AlunoId int NOT NULL REFERENCES dbo.Usuarios(Id),
    ModuloId int NOT NULL REFERENCES dbo.Modulos(Id),
    DataConclusao datetime2 NOT NULL DEFAULT SYSUTCDATETIME(),
    CONSTRAINT UQ_Progresso_Modulo UNIQUE (AlunoId, ModuloId)
);

CREATE TABLE dbo.AcompanhamentosMentor (
    Id int IDENTITY PRIMARY KEY,
    MentorId int NOT NULL REFERENCES dbo.Usuarios(Id),
    AlunoId int NOT NULL REFERENCES dbo.Usuarios(Id),
    Ativo bit NOT NULL DEFAULT 1,
    CONSTRAINT UQ_Acompanhamentos UNIQUE (MentorId, AlunoId),
    CONSTRAINT CK_Acompanhamentos_Pessoas CHECK (MentorId <> AlunoId)
);

CREATE TABLE dbo.OrientacoesMentor (
    Id int IDENTITY PRIMARY KEY,
    MentorId int NOT NULL,
    AlunoId int NOT NULL,
    Texto nvarchar(4000) NOT NULL,
    DataOrientacao datetime2 NOT NULL DEFAULT SYSUTCDATETIME(),
    CONSTRAINT FK_Orientacoes_Vinculo FOREIGN KEY (MentorId, AlunoId) REFERENCES
dbo.AcompanhamentosMentor(MentorId, AlunoId),
    CONSTRAINT CK_Orientacoes_Texto CHECK (LEN(LTRIM(RTRIM(Texto))) > 0)
);

CREATE TABLE dbo.AuditoriaUsuarios (
    Id int IDENTITY PRIMARY KEY,
    UsuarioId int NOT NULL REFERENCES dbo.Usuarios(Id),
    PerfilAnterior nvarchar(20) NOT NULL,
    PerfilNovo nvarchar(20) NOT NULL,
    AtivoAnterior bit NOT NULL,
    AtivoNovo bit NOT NULL,
    DataAlteracao datetime2 NOT NULL DEFAULT SYSUTCDATETIME()
);

-- Índices aceleram buscas e impedem duplicações.
CREATE INDEX IX_Usuarios_PerfilAtivo ON dbo.Usuarios(Perfil, Ativo);
CREATE INDEX IX_Usuarios_Nome ON dbo.Usuarios(Nome);
CREATE INDEX IX_Recuperacoes_Usuario ON dbo.RecuperacoesSenha(UsuarioId, Utilizado, Validade);
CREATE INDEX IX_Sessoes_Usuario ON dbo.Sessoes(UsuarioId, Revogada, Validade);
CREATE INDEX IX_Cursos_AtivoNome ON dbo.Cursos(Ativo, Nome);
CREATE INDEX IX_Materiais_Modulo ON dbo.Materiais(ModuloId, Ativo);
CREATE UNIQUE INDEX UQ_Matriculas_Ativas ON dbo.Matriculas(AlunoId, CursoId) WHERE Ativo = 1;
CREATE INDEX IX_Matriculas_Curso ON dbo.Matriculas(CursoId, Ativo);
CREATE INDEX IX_Avaliacoes_Modulo ON dbo.Avaliacoes(ModuloId, Ativo);
CREATE INDEX IX_Questoes_Avaliacao ON dbo.Questoes(AvaliacaoId, Ativo);
CREATE INDEX IX_Alternativas_Questao ON dbo.Alternativas(QuestaoId);
CREATE UNIQUE INDEX UQ_Alternativas_Correta ON dbo.Alternativas(QuestaoId) WHERE Correta = 1;
CREATE INDEX IX_Tentativas_Aluno ON dbo.TentativasAvaliacao(AlunoId, DataTentativa);
CREATE INDEX IX_Tentativas_Avaliacao ON dbo.TentativasAvaliacao(AvaliacaoId);
CREATE INDEX IX_Respostas_Alternativa ON dbo.RespostasAluno(AlternativaId, QuestaoId);
```

```
CREATE INDEX IX_Progresso_Modulo ON dbo.ProgressoModulos(ModuloId);
CREATE INDEX IX_Acompanhamentos_Aluno ON dbo.AcompanhamentosMentor(AlunoId, Ativo);
CREATE INDEX IX_Orientacoes_Aluno ON dbo.OrientacoesMentor(AlunoId, DataOrientacao);
CREATE INDEX IX_Auditoria_Usuario ON dbo.AuditoriaUsuarios(UsuarioId, DataAlteracao);
GO

CREATE PROCEDURE dbo.sp_RealizarMatricula
  @AlunoId int,
  @CursoId int
AS
BEGIN
  SET NOCOUNT ON;
  SET XACT_ABORT ON;

  BEGIN TRY
    BEGIN TRANSACTION;

    -- O bloqueio serializa matrículas concorrentes do mesmo aluno.
    IF NOT EXISTS (SELECT 1 FROM dbo.Usuarios WITH (UPDLOCK, HOLDLOCK)
                   WHERE Id = @AlunoId AND Perfil = N'Aluno' AND Ativo = 1)
      THROW 51001, 'Aluno inexistente ou inativo.', 1;

    IF NOT EXISTS (SELECT 1 FROM dbo.Cursos WITH (UPDLOCK, HOLDLOCK)
                   WHERE Id = @CursoId AND Ativo = 1)
      THROW 51002, 'Curso inexistente ou inativo.', 1;

    IF EXISTS (SELECT 1 FROM dbo.Matriculas WHERE AlunoId = @AlunoId AND CursoId = @CursoId AND Ativo = 1)
      THROW 51003, 'O aluno já está matriculado neste curso.', 1;

    INSERT dbo.Matriculas (AlunoId, CursoId) VALUES (@AlunoId, @CursoId);
    DECLARE @Id int = CONVERT(int, SCOPE_IDENTITY());

    COMMIT TRANSACTION;
    SELECT @Id AS Id;
  END TRY
  BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    THROW;
  END CATCH
END;
GO

CREATE PROCEDURE dbo.sp_RelatorioCurso
  @CursoId int = NULL
AS
BEGIN
  SET NOCOUNT ON;

  -- Cada subconsulta calcula uma métrica sem multiplicar registros.
  SELECT c.Id AS CursoId, c.Nome,
      (SELECT COUNT(*) FROM dbo.Matriculas mt WHERE mt.CursoId = c.Id AND mt.Ativo = 1) AS Matriculas,
      COALESCE((SELECT AVG(t.Nota) FROM dbo.TentativasAvaliacao t
        JOIN dbo.Avaliacoes a ON a.Id = t.AvaliacaoId JOIN dbo.Modulos m ON m.Id = a.ModuloId
        WHERE m.CursoId = c.Id), 0) AS Media,
      COALESCE((SELECT AVG(CAST(t.Aprovado AS decimal(5,2))) * 100 FROM dbo.TentativasAvaliacao t
        JOIN dbo.Avaliacoes a ON a.Id = t.AvaliacaoId JOIN dbo.Modulos m ON m.Id = a.ModuloId
        WHERE m.CursoId = c.Id), 0) AS Aprovacao,
      COALESCE((SELECT AVG(progresso.Percentual) FROM dbo.Matriculas mt
        CROSS APPLY (SELECT CAST(CASE WHEN COUNT(*) = 0 THEN 0
          ELSE 100.0 * SUM(CASE WHEN p.Id IS NOT NULL THEN 1 ELSE 0 END) / COUNT(*) END AS decimal(5,2)) AS
Percentual
          FROM dbo.Modulos m LEFT JOIN dbo.ProgressoModulos p ON p.ModuloId = m.Id AND p.AlunoId =
mt.AlunoId
          WHERE m.CursoId = c.Id AND m.Ativo = 1) progresso
```

```
            WHERE mt.CursoId = c.Id AND mt.Ativo = 1), 0) AS ProgressoMedio
    FROM dbo.Cursos c
    WHERE @CursoId IS NULL OR c.Id = @CursoId
    ORDER BY c.Nome;
END;
GO

-- Registra somente mudanças de perfil ou de situação.
CREATE TRIGGER dbo.tr_AuditoriaUsuarios
ON dbo.Usuarios
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    INSERT dbo.AuditoriaUsuarios (UsuarioId, PerfilAnterior, PerfilNovo, AtivoAnterior, AtivoNovo)
    SELECT i.Id, d.Perfil, i.Perfil, d.Ativo, i.Ativo
    FROM inserted i JOIN deleted d ON d.Id = i.Id
    WHERE i.Perfil <> d.Perfil OR i.Ativo <> d.Ativo;
END;
GO

-- Confere a estrutura criada, sem adicionar dados fictícios.
COMMIT TRANSACTION;
SELECT name AS Tabela FROM sys.tables ORDER BY name;

-- NOVAS FUNCOES: certificados, chamados e documentos
-- Atualização das novas funções. Execute no banco atual; pode ser repetida.
USE TechLearnAcademy_PIMIV;
GO
SET ANSI_NULLS ON;
SET QUOTED_IDENTIFIER ON;
SET ANSI_PADDING ON;
SET ANSI_WARNINGS ON;
SET ARITHABORT ON;
SET CONCAT_NULL_YIELDS_NULL ON;
SET NUMERIC_ROUNDABORT OFF;
SET XACT_ABORT ON;
BEGIN TRY
    BEGIN TRANSACTION;
    IF COL_LENGTH('dbo.Usuarios','EmailContato') IS NULL
        ALTER TABLE dbo.Usuarios ADD EmailContato nvarchar(254) NULL;
    EXEC(N'UPDATE dbo.Usuarios SET EmailContato=Email WHERE EmailContato IS NULL;');
    IF COL_LENGTH('dbo.Cursos','Excluido') IS NULL
        ALTER TABLE dbo.Cursos ADD Excluido bit NOT NULL DEFAULT 0;
    IF COL_LENGTH('dbo.Modulos','Excluido') IS NULL
        ALTER TABLE dbo.Modulos ADD Excluido bit NOT NULL DEFAULT 0;
    IF COL_LENGTH('dbo.Materiais','Excluido') IS NULL
        ALTER TABLE dbo.Materiais ADD Excluido bit NOT NULL DEFAULT 0;
    IF COL_LENGTH('dbo.Avaliacoes','Excluido') IS NULL
        ALTER TABLE dbo.Avaliacoes ADD Excluido bit NOT NULL DEFAULT 0;
    IF COL_LENGTH('dbo.Materiais','ArquivoNome') IS NULL
        ALTER TABLE dbo.Materiais ADD ArquivoNome nvarchar(255) NULL, ArquivoPdf varbinary(max) NULL;
    IF EXISTS(SELECT 1 FROM sys.key_constraints WHERE name='UQ_Modulos_Ordem')
        ALTER TABLE dbo.Modulos DROP CONSTRAINT UQ_Modulos_Ordem;
    IF NOT EXISTS(SELECT 1 FROM sys.indexes WHERE name='UQ_Modulos_OrdemAtual' AND
object_id=OBJECT_ID('dbo.Modulos'))
        EXEC(N'CREATE UNIQUE INDEX UQ_Modulos_OrdemAtual ON dbo.Modulos(CursoId,Ordem) WHERE
Excluido=0;');
    IF EXISTS(SELECT 1 FROM sys.check_constraints WHERE name='CK_Materiais_Url')
        ALTER TABLE dbo.Materiais DROP CONSTRAINT CK_Materiais_Url;
    EXEC(N'ALTER TABLE dbo.Materiais ADD CONSTRAINT CK_Materiais_Url CHECK
        ((Tipo=N''PDF'' AND ArquivoPdf IS NOT NULL AND DATALENGTH(ArquivoPdf)<=10485760)
         OR (ArquivoPdf IS NULL AND (Url LIKE N''https://%'' OR Url LIKE N''http://%'')));');
    IF OBJECT_ID('dbo.Chamados','U') IS NULL
```

```
  BEGIN
    CREATE TABLE dbo.Chamados(
      Id int IDENTITY PRIMARY KEY,
      SolicitanteId int NOT NULL REFERENCES dbo.Usuarios(Id),
      ResponsavelId int NULL REFERENCES dbo.Usuarios(Id),
      CursoId int NULL REFERENCES dbo.Cursos(Id),
      Tipo nvarchar(30) NOT NULL CHECK(Tipo IN ('AjudaCurso','Ouvidoria','ContatoAdministracao')),
      DestinatarioPerfil nvarchar(20) NOT NULL CHECK(DestinatarioPerfil IN ('Mentor','Gestor','Administrador')),
      Assunto nvarchar(150) NOT NULL,
      Descricao nvarchar(4000) NOT NULL,
      Status nvarchar(20) NOT NULL DEFAULT 'Aberto' CHECK(Status IN ('Aberto','Em atendimento','Encerrado')),
      DataAbertura datetime2 NOT NULL DEFAULT SYSUTCDATETIME(),
      DataEncerramento datetime2 NULL
    );
    CREATE INDEX IX_Chamados_Fila ON dbo.Chamados(DestinatarioPerfil,Status,ResponsavelId);
    CREATE INDEX IX_Chamados_Solicitante ON dbo.Chamados(SolicitanteId,CursoId);
  END;
  IF OBJECT_ID('dbo.MensagensChamado','U') IS NULL
    CREATE TABLE dbo.MensagensChamado(
      Id int IDENTITY PRIMARY KEY,
      ChamadoId int NOT NULL REFERENCES dbo.Chamados(Id),
      UsuarioId int NOT NULL REFERENCES dbo.Usuarios(Id),
      Texto nvarchar(4000) NOT NULL,
      DataEnvio datetime2 NOT NULL DEFAULT SYSUTCDATETIME()
    );
  IF OBJECT_ID('dbo.SolicitacoesCertificado','U') IS NULL
  BEGIN
    CREATE TABLE dbo.SolicitacoesCertificado(
      Id int IDENTITY PRIMARY KEY,
      AlunoId int NOT NULL REFERENCES dbo.Usuarios(Id),
      CursoId int NOT NULL REFERENCES dbo.Cursos(Id),
      Status nvarchar(20) NOT NULL DEFAULT 'Pendente' CHECK(Status IN ('Pendente','Aprovado','Recusado')),
      GestorId int NULL REFERENCES dbo.Usuarios(Id),
      Motivo nvarchar(1000) NULL,
      Codigo varchar(64) NULL,
      NomeAluno nvarchar(100) NULL,
      NomeCurso nvarchar(150) NULL,
      CargaHoraria int NULL,
      DataSolicitacao datetime2 NOT NULL DEFAULT SYSUTCDATETIME(),
      DataAprovacao datetime2 NULL,
      CONSTRAINT UQ_Certificado_AlunoCurso UNIQUE(AlunoId,CursoId)
    );
    CREATE UNIQUE INDEX UQ_Certificado_Codigo ON dbo.SolicitacoesCertificado(Codigo) WHERE Codigo IS NOT
NULL;
  END;
  IF EXISTS(SELECT 1 FROM sys.foreign_keys WHERE name='FK_Orientacoes_Vinculo')
    ALTER TABLE dbo.OrientacoesMentor DROP CONSTRAINT FK_Orientacoes_Vinculo;
  -- A tabela de vínculos antigos permanece como histórico, sem uso pela aplicação.
  EXEC(N'CREATE OR ALTER TRIGGER dbo.tr_LimiteModulos ON dbo.Modulos AFTER INSERT AS
  BEGIN
    SET NOCOUNT ON;
    IF EXISTS(SELECT 1 FROM dbo.Modulos m JOIN inserted i ON i.CursoId=m.CursoId
          WHERE m.Excluido=0 GROUP BY m.CursoId HAVING COUNT(DISTINCT m.Id)>4)
      THROW 51004, ''O curso pode ter no máximo quatro módulos.'', 1;
  END;');
  -- Conclusões antigas sem aprovação não liberam certificados.
  EXEC(N'DELETE p FROM dbo.ProgressoModulos p WHERE
    NOT EXISTS(SELECT 1 FROM dbo.Avaliacoes a WHERE a.ModuloId=p.ModuloId AND a.Ativo=1 AND a.Excluido=0)
    OR EXISTS(SELECT 1 FROM dbo.Avaliacoes a OUTER APPLY
      (SELECT TOP(1) t.Aprovado FROM dbo.TentativasAvaliacao t WHERE t.AvaliacaoId=a.Id AND
t.AlunoId=p.AlunoId ORDER BY t.Id DESC) ultima
      WHERE a.ModuloId=p.ModuloId AND a.Ativo=1 AND a.Excluido=0 AND COALESCE(ultima.Aprovado,0)=0);');
  COMMIT TRANSACTION;
END TRY
```

```
BEGIN CATCH
   IF @@TRANCOUNT>0 ROLLBACK TRANSACTION;
   THROW;
END CATCH;
GO
SELECT 'Atualização concluída.' AS Mensagem;

-- Correções de recuperação e avaliações de 11/10/2026.
-- Execute no banco utilizado pelo site. Não apaga notas nem matrículas.
SET ANSI_NULLS ON;
SET QUOTED_IDENTIFIER ON;
SET ANSI_PADDING ON;
SET ANSI_WARNINGS ON;
SET ARITHABORT ON;
SET CONCAT_NULL_YIELDS_NULL ON;
SET NUMERIC_ROUNDABORT OFF;
SET XACT_ABORT ON;
BEGIN TRY
   BEGIN TRANSACTION;
   IF COL_LENGTH('dbo.Usuarios','EmailContato') IS NULL
      THROW 51000, 'Execute primeiro Banco/05-atualizar-funcionalidades.sql.', 1;
   IF EXISTS(SELECT EmailContato FROM dbo.Usuarios WHERE EmailContato IS NOT NULL GROUP BY EmailContato
HAVING COUNT(*)>1)
      THROW 51000, 'Há e-mails pessoais repetidos. Edite os usuários para informar um contato diferente em cada
conta antes desta atualização.', 1;
   IF NOT EXISTS(SELECT 1 FROM sys.indexes WHERE object_id=OBJECT_ID('dbo.Usuarios') AND
name='UQ_Usuarios_EmailContato')
      EXEC(N'CREATE UNIQUE INDEX UQ_Usuarios_EmailContato ON dbo.Usuarios(EmailContato) WHERE
EmailContato IS NOT NULL;');

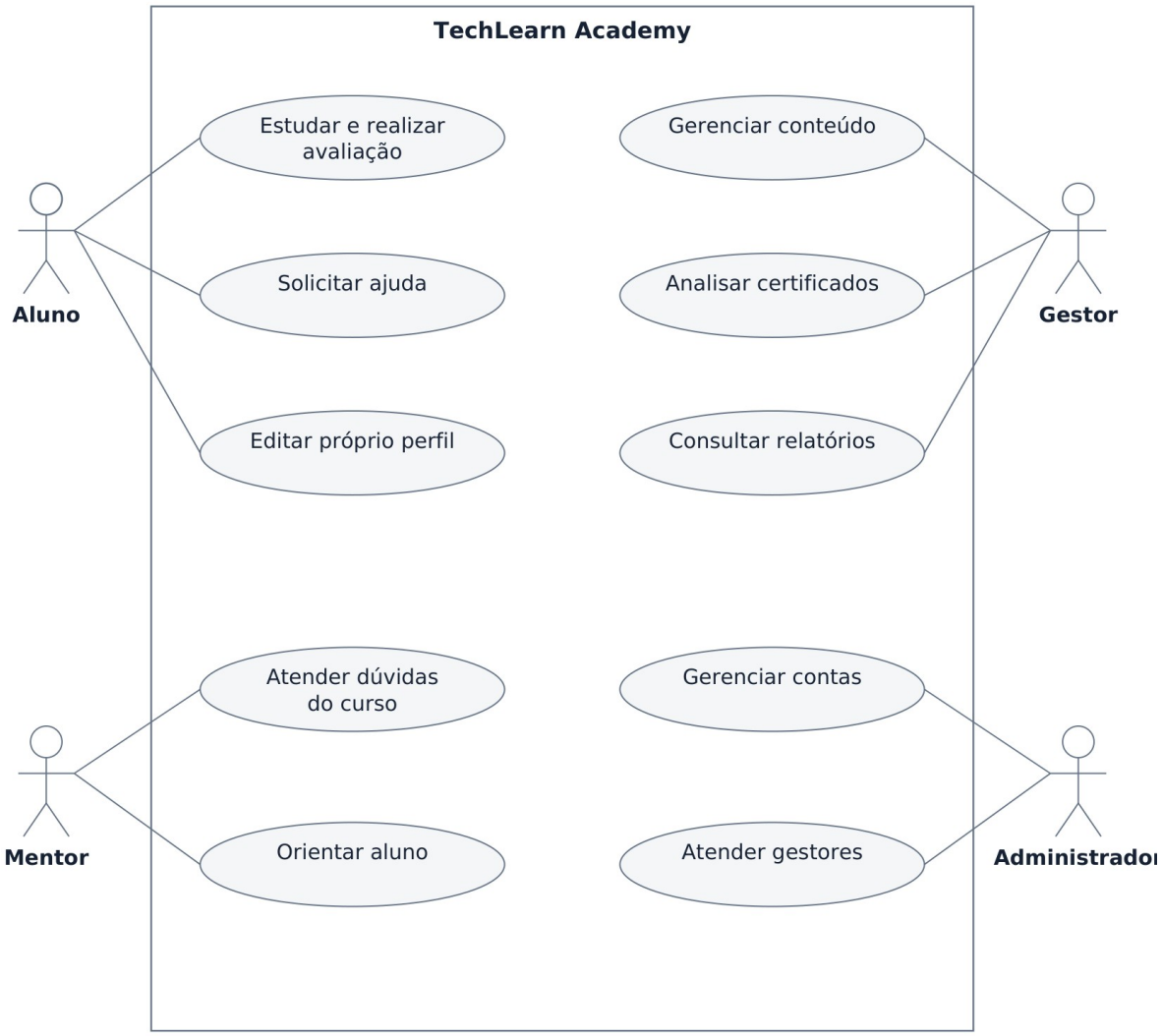
   -- A tentativa começa em 1. Reiniciar o curso aumenta este número.
   IF COL_LENGTH('dbo.Matriculas','TentativaCurso') IS NULL
      ALTER TABLE dbo.Matriculas ADD TentativaCurso int NOT NULL CONSTRAINT DF_Matriculas_TentativaCurso
DEFAULT 1 CONSTRAINT CK_Matriculas_TentativaCurso CHECK(TentativaCurso>0);
   IF COL_LENGTH('dbo.TentativasAvaliacao','TentativaCurso') IS NULL
      ALTER TABLE dbo.TentativasAvaliacao ADD TentativaCurso int NOT NULL CONSTRAINT
DF_Tentativas_TentativaCurso DEFAULT 1 CONSTRAINT CK_Tentativas_Curso CHECK(TentativaCurso>0);
   IF NOT EXISTS(SELECT 1 FROM sys.indexes WHERE object_id=OBJECT_ID('dbo.TentativasAvaliacao') AND
name='IX_Tentativas_CursoAtual')
      EXEC(N'CREATE INDEX IX_Tentativas_CursoAtual ON
dbo.TentativasAvaliacao(AlunoId,AvaliacaoId,TentativaCurso);');
   -- Notas anteriores ficam na tentativa 1. O servidor bloqueia novos envios
   -- de avaliações já respondidas, preservando o histórico que já existia.
   COMMIT TRANSACTION;
END TRY
BEGIN CATCH
   IF @@TRANCOUNT>0 ROLLBACK TRANSACTION;
   THROW;
END CATCH;
GO
```

APÊNDICE B MODELAGEM UML DA SOLUÇÃO

B.1 Casos de uso por perfil

A versão de apresentação é Diagramas/TechLearnAcademy-Diagramas-simples.asta, editável no Astah, com 14 casos de uso e 44 PNGs exportados em Diagramas/Imagens. Inclui os quatro tipos UML; certificados e relatórios não têm caso individual. Cada associação indica participação no objetivo, não acesso irrestrito. Sessão e perfil são conferidos no servidor (OBJECT MANAGEMENT GROUP, 2017).

Figura 10 - Casos de uso principais por perfil

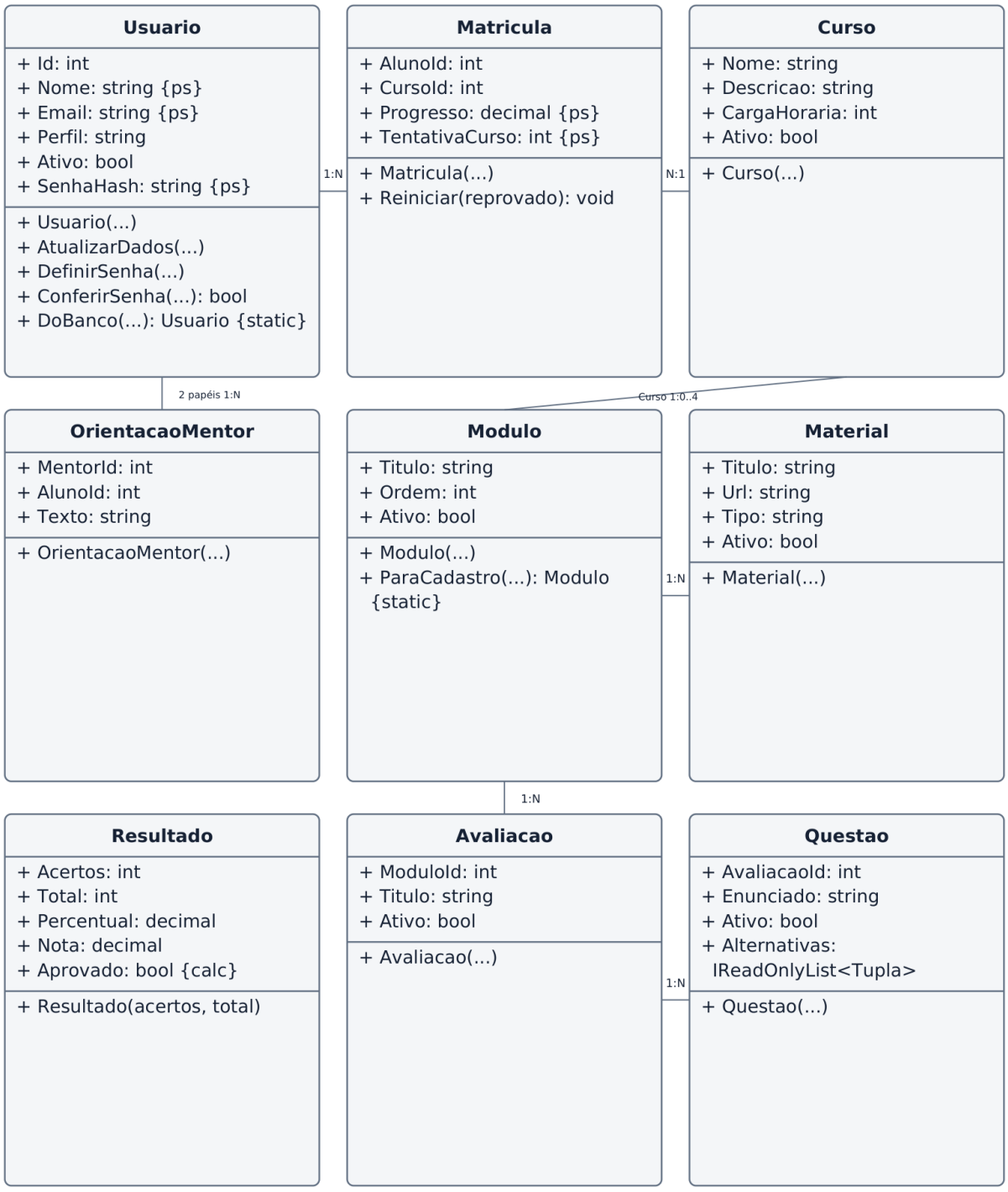


Fonte: elaboração dos autores (2026).

As permissões são conferidas em UsuariosController, CursosController, AvaliacoesController e GestaoController e, quando necessário, nos serviços. O papel Administrador não recebe automaticamente a função de criar curso, exclusiva do Gestor. O conjunto é um recorte de apresentação, não um inventário exaustivo de todas as rotas.

B.2 Classes centrais com atributos e operações

Figura 11 - Nove classes centrais do backend C#



Fonte: elaboração dos autores (2026).

Legenda: + indica público; {ps}, private set; {calc}, propriedade calculada; {static}, membro estático. Construtores e operações com (...) têm parâmetros omitidos para legibilidade. Demais propriedades de leitura são definidas na construção. Tupla representa Texto e Correta. As linhas sintetizam vínculos de negócio guardados por IDs; o limite de quatro considera módulos não excluídos.

B.3 Conferência das classes com a persistência

Classe	Propriedades e operações relevantes	Correspondência com o SQL
Usuario	Id, Nome, Email, Telefone, Sexo, SexoOutro, Perfil, Ativo, SenhaHash; atualizar dados pessoais e senha.	Usuarios; EmailContato e datas são tratados fora deste modelo central.
Curso	Nome, Descricao, CargaHoraria, Ativo; construtor.	Cursos; o Id da tabela não é propriedade da classe.
Modulo	Titulo, Ordem, Ativo; construtor e ParaCadastro.	Modulos; CursoId é recebido no pedido/serviço e enviado ao SQL.
Material	Titulo, Url, Tipo, Ativo; construtor.	Materiais; ModuloId, bytes e nome do PDF ficam no serviço e na tabela.
Matricula	AlunoId, CursoId, Progresso, TentativaCurso; Reiniciar.	Matriculas; progresso também deriva de ProgressoModulos.
Avaliacao	ModuloId, Titulo, Ativo; construtor.	Avaliacoes; Id gerenciado pela persistência e pelas rotas.
Questao	AvaliacaoId, Enunciado, Ativo, Alternativas; construtor.	Questoes e Alternativas; tuplas no objeto e linhas separadas no banco.
Resultado	Acertos, Total, Percentual, Nota, Aprovado; construtor.	Cálculo gravado em TentativasAvaliacao, sem tabela Resultado.
OrientacaoMentor	MentorId, AlunoId, Texto; construtor.	OrientacoesMentor; vínculo autorizado pelo serviço.

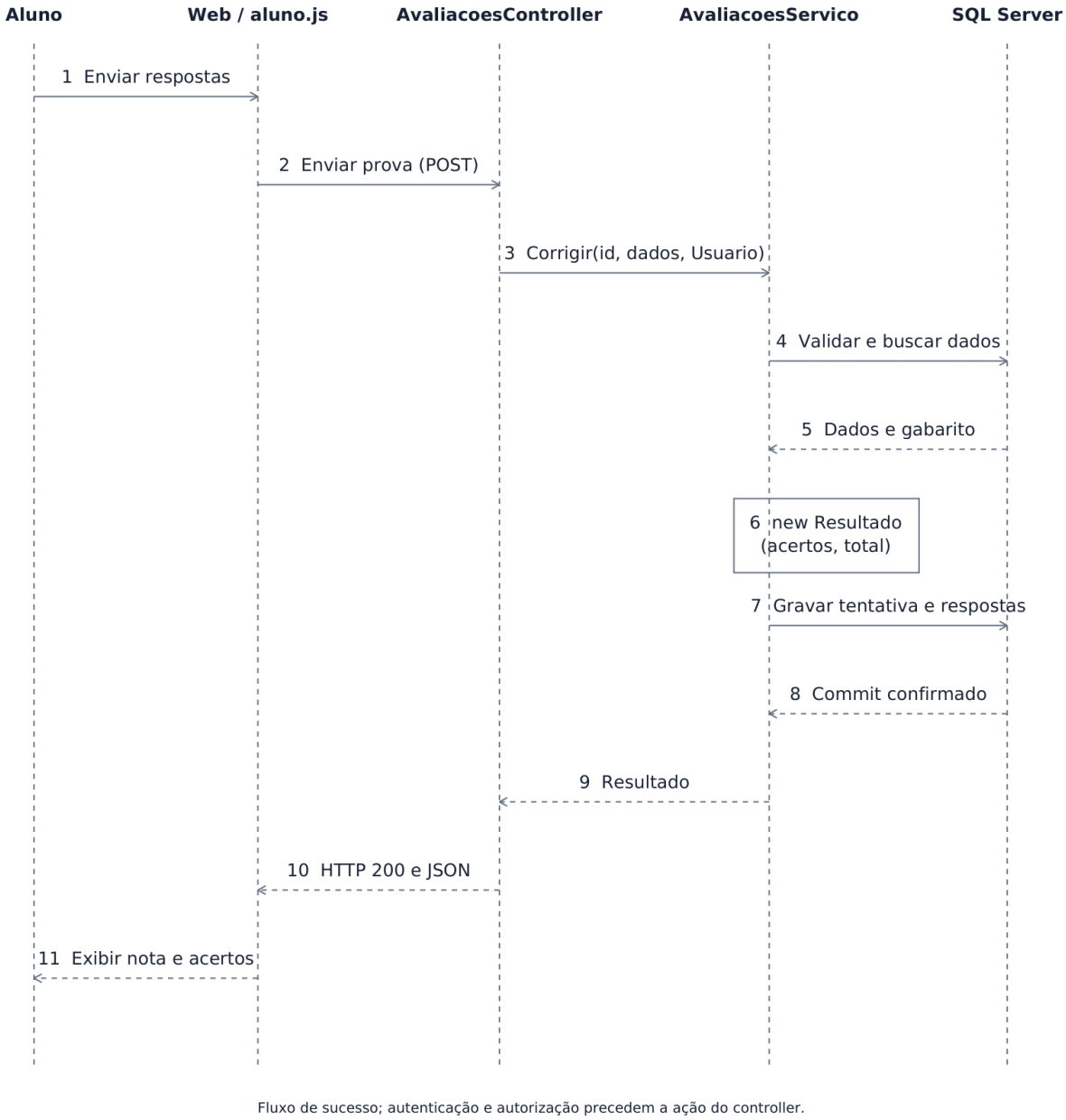
Fonte: elaboração dos autores (2026).

Origem dos tipos: TechLearnAcademy.Api/Modelos, arquivos homônimos. As nove classes não incluem os tipos Dados de Pedidos.cs, serviços, controllers ou modelos de resposta do desktop. Não há subclasses para Aluno, Mentor, Gestor e Administrador. Resultado não conserva referências a questão ou avaliação; recebe números apurados pelo serviço.

A associação desenhada expressa o vínculo persistido, não a existência de propriedades de navegação como Curso.Modulos. Curso não tem Id e Modulo não tem CursoId. Essas distinções permitem comparar a modelagem ao C# sem exigir equivalência integral entre classe e tabela.

B.4 Sequência da realização de avaliação

Figura 12 - Sequência de envio e correção da avaliação

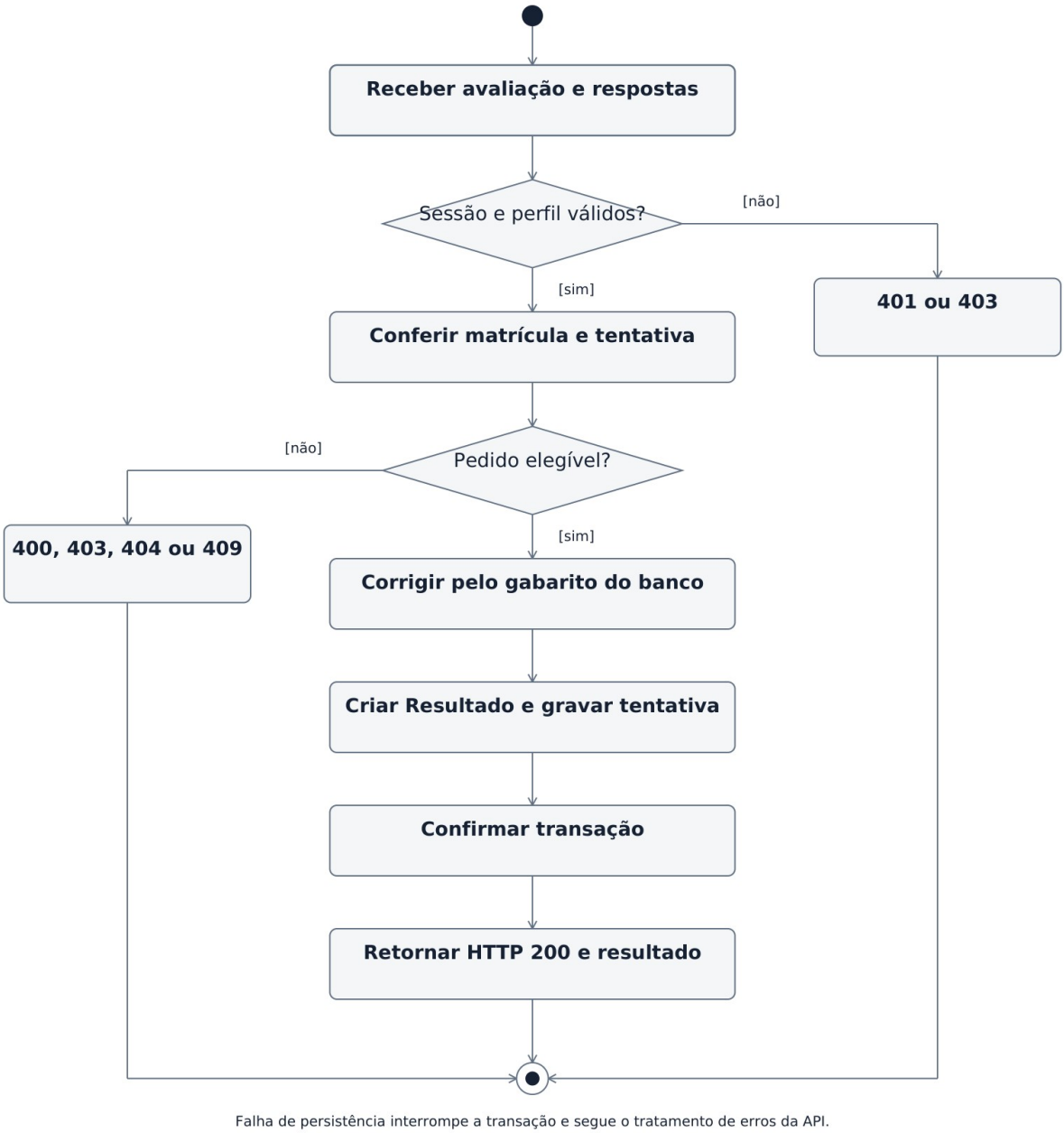


Fonte: elaboração dos autores (2026).

AvaliacoesController recebe POST /api/avaliacoes/{id:int}/respostas e aciona Corrigir. AvaliacoesServico valida tentativa e respostas, consulta o gabarito, cria Resultado e grava tentativa, respostas e progresso na transação. Em sucesso, o controller retorna HTTP 200 com id, acertos, total, percentual, nota e aprovado. A interface apresenta os valores calculados no servidor.

B.5 Atividade da realização de avaliação

Figura 13 - Atividade e decisões na correção da avaliação



Fonte: elaboração dos autores (2026).

Pedido elegível significa avaliação disponível, matrícula e acesso válidos, tentativa atual, ausência de envio anterior impeditivo e conjunto completo de respostas. Cada alternativa precisa pertencer à questão. Uma recusa encerra o fluxo antes da gravação de sucesso. Os números HTTP dependem da regra violada e do middleware.

APÊNDICE C REGISTROS DOS TESTES DE USABILIDADE

C.1 Procedimento e tarefas aplicadas

A atividade ocorreu no laboratório de informática da UNIP Campus Dutra, em São José dos Campos, durante o Dia da UNIP. O relato registra três participantes externos ao desenvolvimento, utilizando o perfil de Aluno e consultando um roteiro impresso. Os integrantes acompanharam a navegação e esclareceram o objetivo das atividades. Não foi informado um procedimento de cronometragem ou escala padronizada.

Tarefa	Ação descrita no roteiro aplicado	Critério observável
T1 Ajuda ao mentor	Acessar Ajuda e ouvidoria; relacionar curso; informar assunto e mensagem; enviar chamado.	Solicitação registrada e confirmação compreensível.
T2 Avaliação	Acessar curso em que há matrícula; escolher avaliação; responder, enviar e consultar resultado.	Resultado apresentado com nota, acertos e percentual.
T3 Perfil	Acessar Meu perfil; alterar um dado permitido e salvar.	Dado atualizado e confirmação apresentada.

Fonte: roteiro e relato da atividade extensionista do grupo (2026).

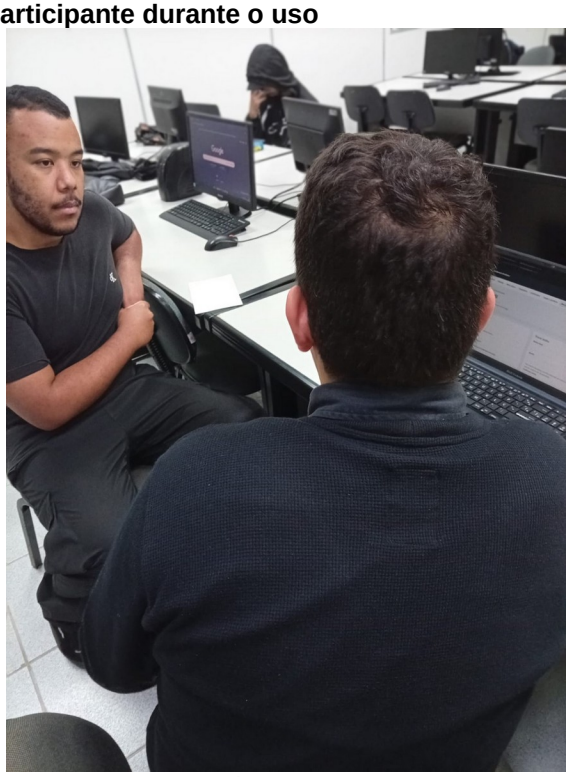
O registro informa conclusão das três tarefas pelos participantes e impressões favoráveis sobre organização, concentração do suporte e apresentação do resultado. As sugestões mencionam visibilidade de comandos, mensagens de confirmação e primeiro acesso. Não há ficha individual suficiente para atribuir cada comentário a uma pessoa ou quantificar dificuldades por tarefa.

O roteiro aplicado contém indicações de navegação. Em um novo ciclo, cenários contextualizados com menor indicação de menus podem avaliar melhor a descoberta das funções. Essa recomendação não altera retroativamente o procedimento executado. Recomenda-se registrar ajuda recebida, erros observados, tempo quando medido e versão do sistema.

C.2 Evidências de utilização e acompanhamento

Figura 14 - Participante utilizando a plataforma e acompanhamento do teste

Participante durante o uso



Acompanhamento pelos integrantes



Fonte: registros fotográficos da atividade extensionista, fornecidos pelo grupo (2026).

Os registros mostram utilização da plataforma e acompanhamento durante a atividade. A imagem confirma o contexto observado; a conclusão das tarefas é descrita no relato do grupo. Não é possível deduzir tempo de execução, autonomia ou taxa de erro apenas pelas fotografias.

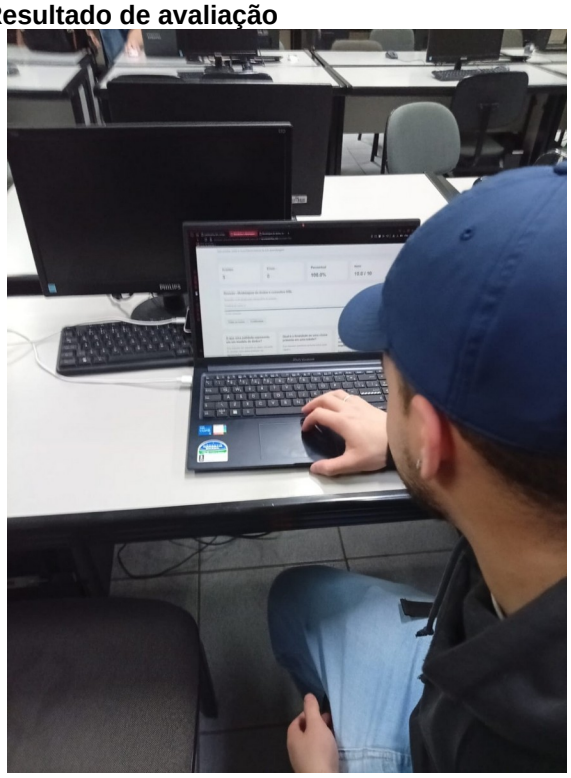
C.3 Evidências das tarefas acadêmicas

Figura 15 - Ajuda ao mentor e consulta do resultado da avaliação

Tarefa de ajuda e ouvidoria



Resultado de avaliação



Fonte: registros da atividade extensionista, fornecidos pelo grupo (2026).

As imagens foram preservadas como evidências do uso observado. A alteração de perfil está descrita no roteiro e no relato, sem captura individual específica nessa seleção. Não se atribuem conclusões sobre todas as funcionalidades, perfis ou dispositivos a essas duas tarefas.

APÊNDICE D RASTREABILIDADE E PENDÊNCIAS DE ACEITE

Parte	Evidência disponível	Verificação ainda necessária
Organização e oferta gratuita	Caracterização, proposta gratuita e simulação mensal de R$ 350,00 com premissas explícitas.	Cotar os serviços, aprovar o orçamento e definir responsáveis pelo custeio antes da abertura pública.
Responsabilidade social	Diretrizes de diversidade, atendimento, acessibilidade e registro dos novos campos de cadastro.	Testar tecnologias assistivas e justificar ou dispensar telefone e gênero antes da abertura pública.
Web e API	Fluxos, rotas, autenticação, registros históricos, conferência da publicação em 07/11 e teste unitário xUnit da confirmação de senha.	Conferir a versão publicada, entregar os registros citados e executar o teste unitário no Visual Studio, preservando a captura do resultado.
Mobile	Descrição do APK Java e capturas reais de acesso e painel.	Fontes Java e relatório M01 a M06, por aparelho e versão.
Desktop	Executável Windows Forms e código C# reconstruído do binário 1.0.2.	Comparar binário recompilado com o distribuído; a reconstrução não é o fonte original.
POO e UML	Nove classes, quatro tipos de diagrama, Astah editável e 44 PNGs dos 14 casos.	Aceite do grupo e conferência com os fontes da entrega final.
Banco	19 tabelas no SQL; MER, dicionários, procedures, triggers e novos campos de Usuarios.	Validar instalação ou atualização em base isolada e documentar o resultado.
Nuvem e DevOps	Publicação histórica, Dockerfile descrito e plano operacional.	Demonstrar automação, monitoramento e restauração quando executados.
Gestão e usabilidade	Backlogs, cronograma, Kanban, tarefas e relato de três participantes.	Aceite final e registros mais específicos em um novo ciclo de uso.

Fonte: elaboração dos autores (2026).

Ramo main em 81f2d94 (08/11/2026): 295 arquivos, incluindo fontes da API e web, nove scripts SQL, Astah, 50 PNGs, seis SVGs, a modelagem editável do banco em Draw.io, o projeto xUnit e o C# desktop reconstruído. Desde 94da0a8, quinze commits acrescentaram o teste unitário, a modelagem do banco e atualizações documentais, sem modificar os fluxos funcionais da API, da web ou do desktop. Esta revisão não executou programas. APK e fontes Java/Gradle não estão no Git; MAUI é legado, e os binários são distribuídos pelo Azure. Relatórios anteriores citam integracao.json, adicionais.json, testes.json, colagem.json e azure_revisao.json, ausentes da árvore. O PDF do Git foi atualizado em 08/11; o inventário de arquivos, por si só, não comprova execução nem aprovação dos testes.

D.1 Protocolo de homologação do aplicativo Android

Executar M01 a M06 no APK distribuído e registrar data, modelo do aparelho, versão do Android e versão/hash do APK. Para cada caso, informar conta/perfil de teste, estado inicial, passos, resultado observado, captura ou log, responsável e status (aprovado, reprovado ou bloqueado). Uma captura de tela isolada não substitui a execução da jornada.

Quadro 25 - Passos e resultados esperados para M01 a M06

Caso	Procedimento	Resultado esperado e situação atual
M01	Instalar e abrir; autenticar Aluno e outro perfil autorizado.	Abre sem erro e exibe painel correto. Capturas de login e painel do Aluno disponíveis; aparelho/versão não registrados.
M02	Entrar, sair e tentar recurso restrito com sessão encerrada ou outro perfil.	API recusa acesso indevido e exige novo login. Resultado pendente de relatório.
M03	Consultar catálogo, matricular-se e abrir módulos; conferir a matrícula na web.	A mesma matrícula aparece nos canais sem duplicação. Resultado pendente.
M04	Abrir PDF autorizado; tentar arquivo sem matrícula; voltar ao módulo.	Leitura permitida ao matriculado, recusa ao não autorizado e navegação funcional. Pendente.
M05	Enviar respostas de avaliação e consultar nota, acertos e progresso na web.	Cálculo consistente e uma tentativa gravada sem duplicação. Resultado pendente.
M06	Desligar rede durante consulta/envio; repetir com sessão expirada.	Erro claro, sem sucesso fictício nem envio duplicado; reautenticação quando necessária. Pendente.

A descrição dos passos é um roteiro de teste, não um registro de execução. Para concluir o aceite, anexar evidências identificáveis e preencher o resultado observado individualmente em cada caso.

D.2 Conferência dos fontes e binários

Quadro 26 - Correspondência necessária entre entregas e fontes

Entrega	Evidência atual	Conferência para aceite
API e web	Arquitetura, rotas e fonte examinados no relatório; teste unitário xUnit da confirmação de senha implementado.	Registro de conferência da publicação em 07/11 disponível; executar o teste unitário, guardar a captura e repetir as verificações após mudanças.
Android APK	Pacote e capturas de acesso e painel examinados.	Reunir projeto Java/Gradle correspondente, compilar APK e comparar versão e funções com o binário entregue.
Desktop EXE	Executável 1.0.2 e código C# nativo reconstruído por descompilação; WebView2 é legado.	Comparar o binário recompilado com a entrega e documentar diferenças da reconstrução.
Banco SQL	Script integral no Apêndice A e dicionários citados.	Executar o .sql já presente em base isolada e registrar o resultado, sem atingir produção.

O desktop nativo foi reconstruído do executável e, conforme o registro do projeto, compilou em 27 de outubro; isso não recupera o fonte original nem comprova equivalência do novo binário. Os arquivos C# da reconstrução estão no repositório, enquanto o EXE e o APK são distribuídos pela página de aplicativos. Para Android, faltam projeto Java/Gradle e resultados M01 a M06 por aparelho. O grupo deve registrar versão, hash e comparação da entrega com os fontes disponíveis.

D.3 Enquadramento de JavaScript no mobile

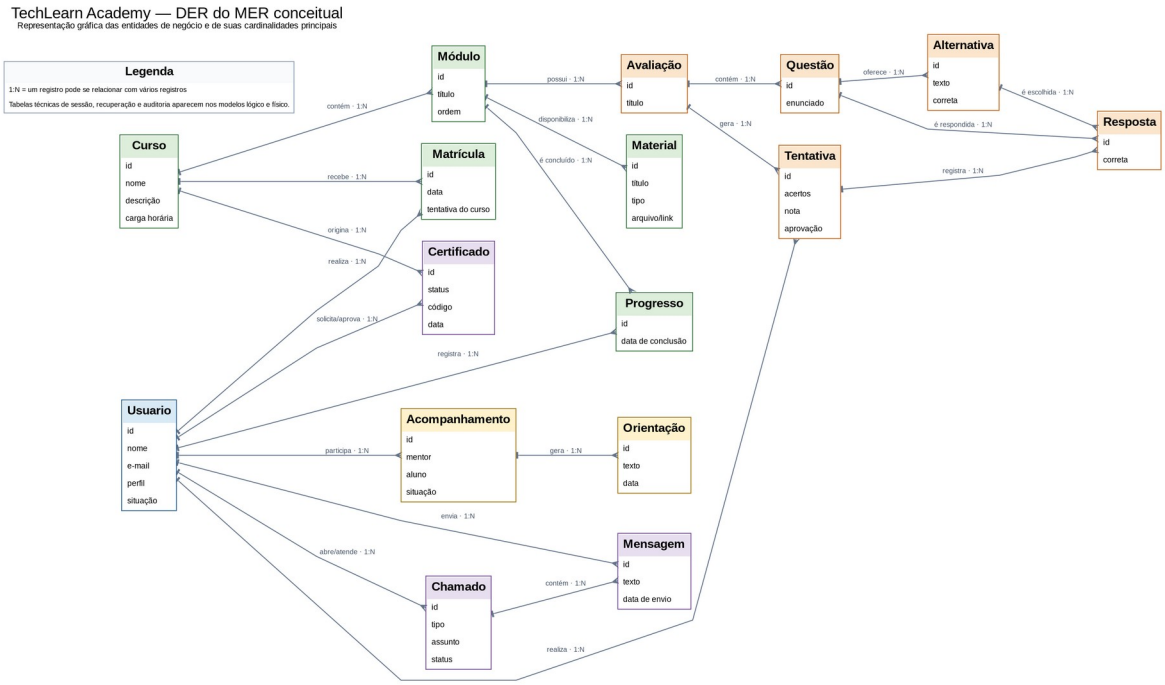
A versão móvel documentada é Android nativo em Java; JavaScript integra o cliente web responsivo. A etapa móvel deste PIM pede aplicativo integrado, sem linguagem obrigatória explicitada na proposta geral examinada. Caso a avaliação específica da disciplina Desenvolvimento Mobile em JavaScript exija código móvel em JavaScript, a versão atual não apresenta essa evidência e o critério deve ser confirmado com o docente.

APÊNDICE E MODELAGEM COMPLETA DO BANCO DE DADOS

Este apêndice reúne o Diagrama Entidade-Relacionamento (DER), o modelo lógico e o modelo físico produzidos a partir da cópia do banco TechLearn Academy exportada do Azure SQL em 7 de novembro de 2026. O inventário confirmou 19 tabelas, 127 colunas e 27 chaves estrangeiras. O BACPAC foi adotado como fonte principal por representar a estrutura efetivamente implantada.

E.1 Diagrama do modelo entidade-relacionamento

Figura 16 - DER do MER conceitual da TechLearn Academy

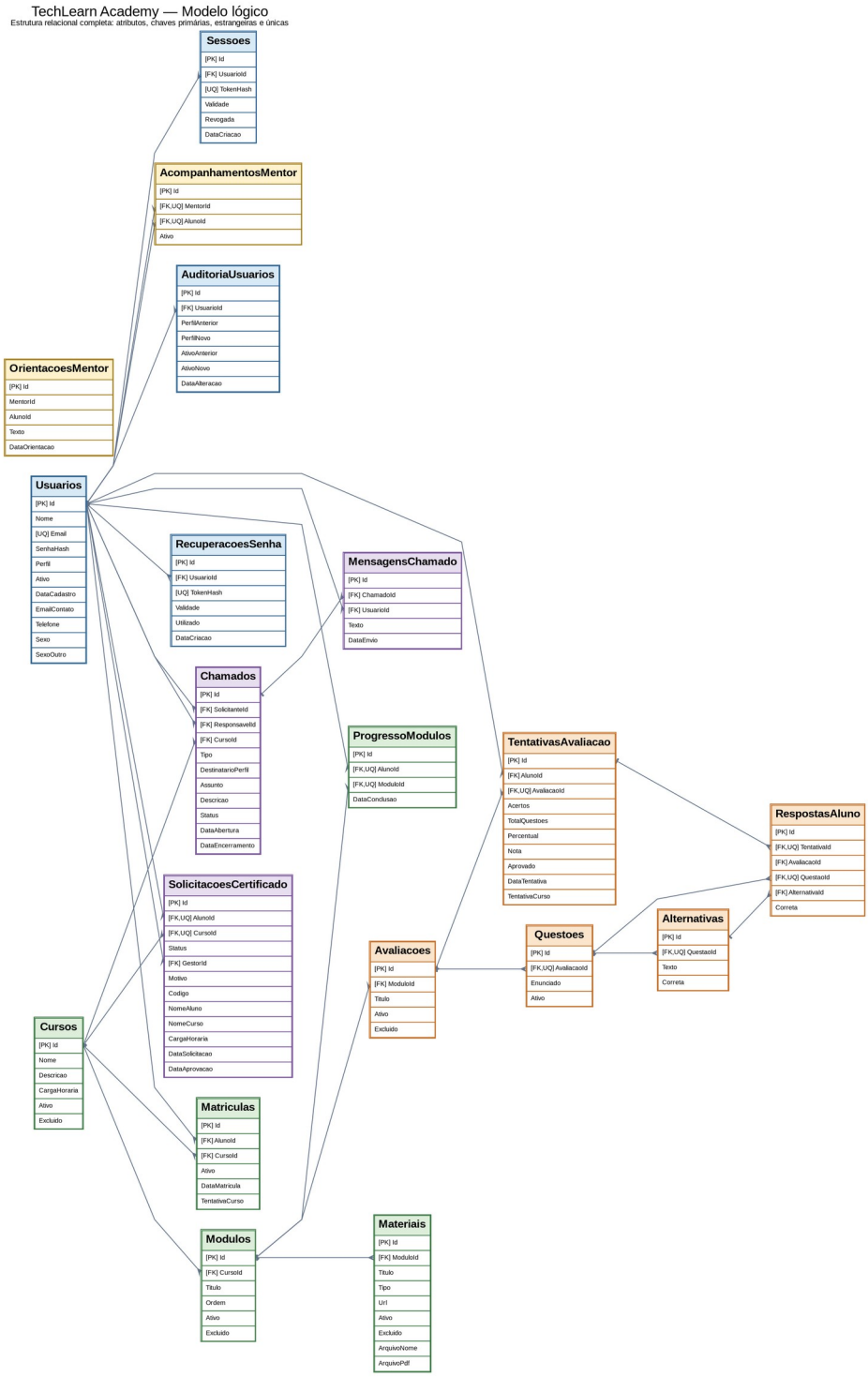


Fonte: elaboração dos autores (2026), com base no banco exportado do Azure SQL.

O DER apresenta as entidades de negócio e as cardinalidades principais. Tabelas técnicas de sessão, recuperação de senha e auditoria são detalhadas nos modelos lógico e físico, evitando sobrecarregar a visão conceitual.

E.2 Modelo lógico

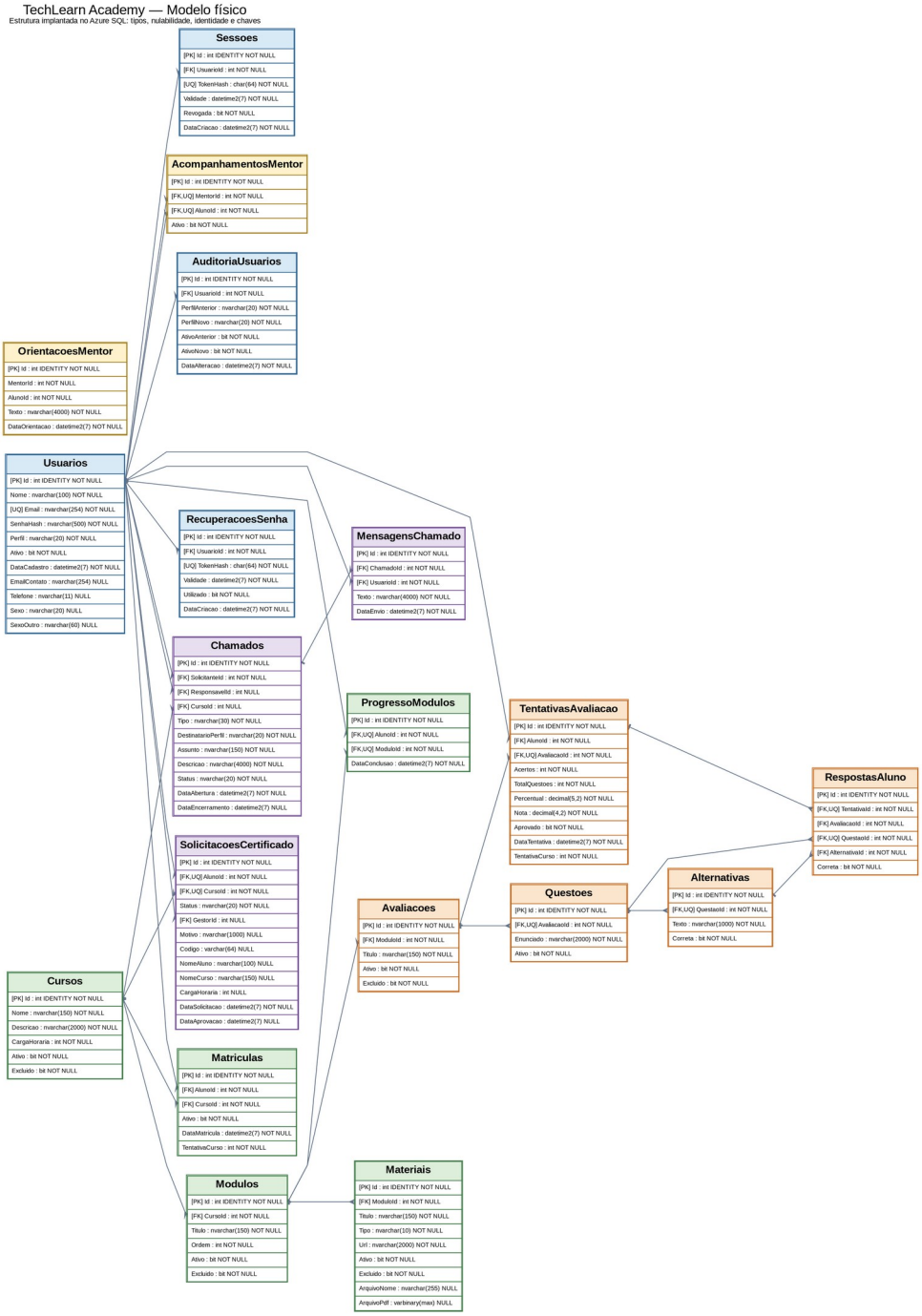
Figura 17 - Modelo lógico completo do banco de dados



Fonte: elaboração dos autores (2026), com base no banco exportado do Azure SQL.

E.3 Modelo físico

Figura 18 - Modelo físico completo do banco de dados



Fonte: elaboração dos autores (2026), com base no banco exportado do Azure SQL.

No modelo físico, PK indica chave primária, FK indica chave estrangeira, UQ indica participação em restrição única, IDENTITY indica numeração automática e NULL/NOT NULL informa a obrigatoriedade de armazenamento. A versão editável em Draw.io e as três exportações SVG estão versionadas no Git; as figuras 16, 17 e 18 incorporam os diagramas ao PIM para a entrega acadêmica em arquivo único.